

云原生训练营

模块十：Kubernetes 的生产化运维

孟凡杰

前 eBay 资深架构师



目录

- 镜像安全
- 基于Kubernetes的DevOps
- 基于GitHub Action的自动化流水线
- Tekton
- ArgoCD
- 监控和日志

1. 镜像仓库

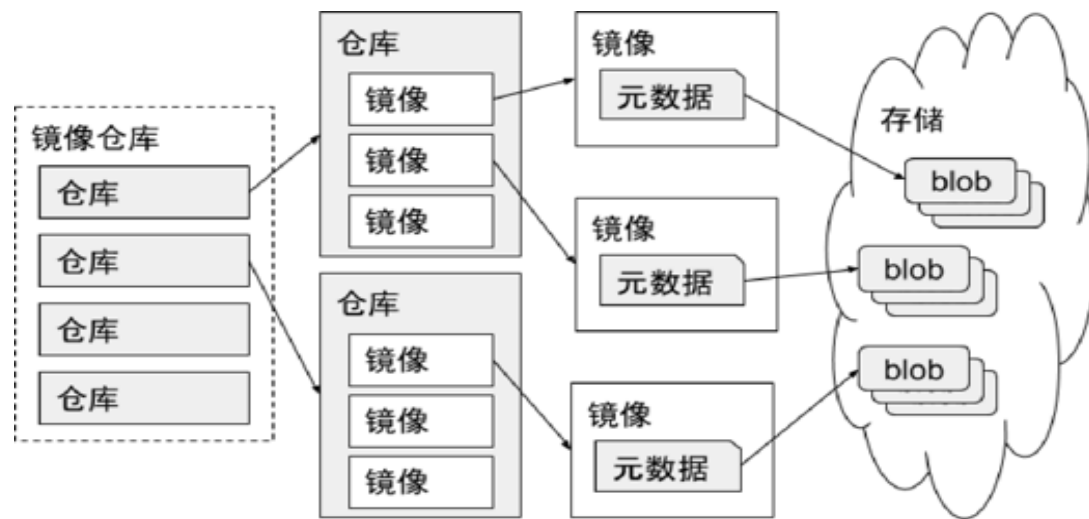
镜像仓库

镜像仓库（Docker Registry）负责存储、管理和分发镜像。

镜像仓库管理多个Repository，Repository通过命名来区分。每个Repository包含一个或多个镜像，镜像通过镜像名称和标签（Tag）来区分。

客户端拉取镜像时，要指定三要素：

- 镜像仓库：要从哪一个镜像仓库拉取镜像，通常通过DNS或IP地址来确定一个镜像仓库，如hub.docker.com
- Repository: 组织名，如cncamp
- 镜像名称+标签：如nginx:latest



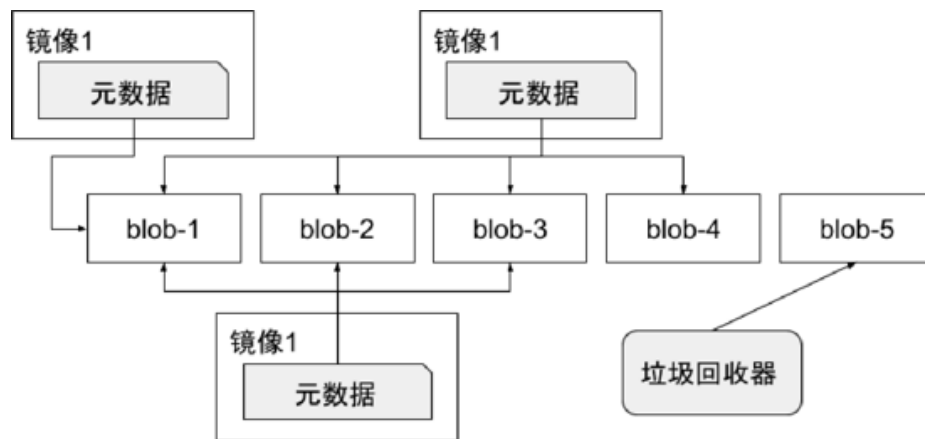
镜像仓库遵循OCI的Distribution Spec

HTTP Verb	URL	功能
GET	/v2/	检查镜像仓库实现的规范、版本
GET	/v2/_catalog	获取仓库列表
GET	/v2/<name>/tags/list	获取一个仓库下所有的标签
PUT	/v2/<name>/manifests/<reference>	上传镜像manifest信息
DELETE	/v2/<name>/manifests/<reference>	删除镜像
GET	/v2/<name>/manifests/<reference>	获取一个镜像的manifest信息
GET	/v2/<name>/blobs/<digest>	获取一个镜像的文件层
POST	/v2/<name>/blobs/uploads/	启动一个镜像的上传
PUT	/v2/<name>/blobs/uploads/<session_id>	结束文件层上传

数据和块文件

镜像由元数据和块文件两部分组成，镜像仓库的核心职能就是管理这两项数据。

- 元数据：
 - 元数据用于描述一个镜像的核心信息，包含镜像的镜像仓库、仓库、标签、校验码、文件层、镜像构建描述等信息。
 - 通过这些信息，可以从抽象层面完整地描述一个镜像：它是如何构建出来的、运行过什么构建命令、构建的每一个文件层的校验码、打的标签、镜像的校验码等。
- 块文件 (blob)
 - 块文件是组成镜像的联合文件层的实体，每一个块文件是一个文件层，内部包含对应文件层的变更。



镜像仓库

公有镜像仓库优势

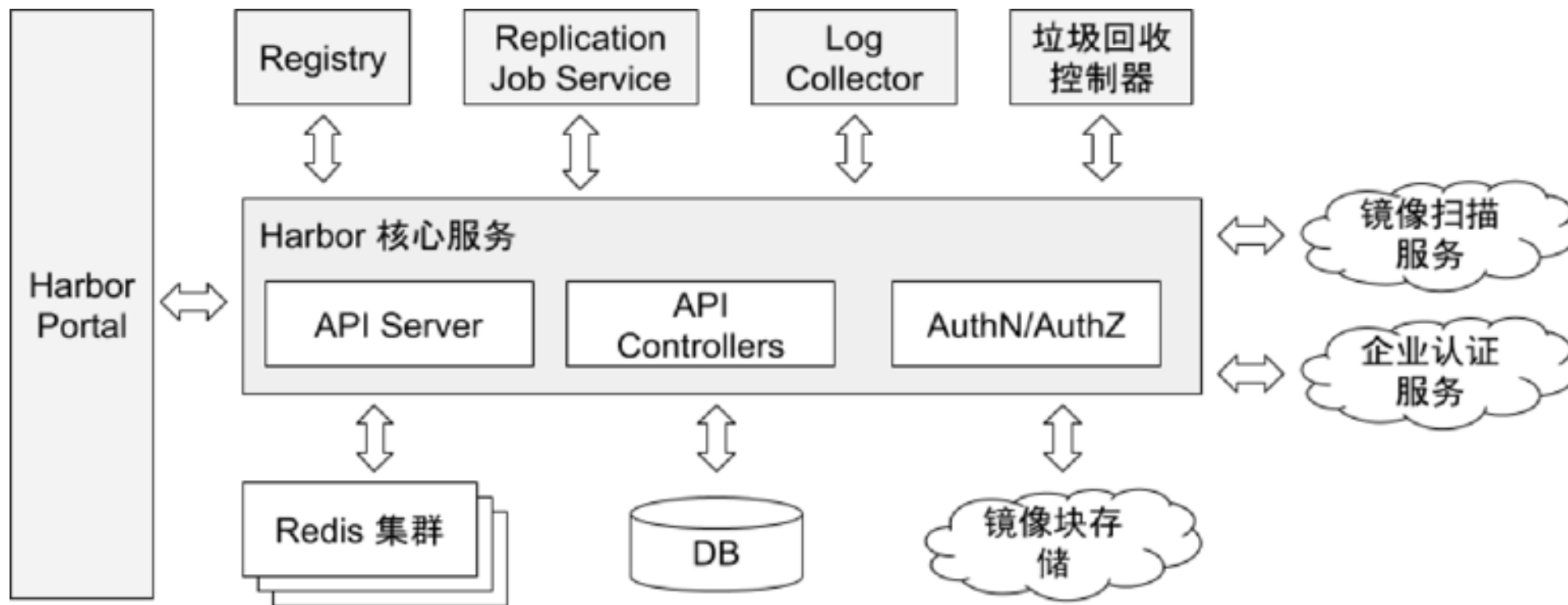
- 开放：任何开发者都可以上传、分享镜像到公有镜像仓库中。
- 便捷：开发者可以非常方便地搜索、拉取其他开发者的镜像，避免重复造轮子。
- 免运维：开发者只需要关注应用开发，不必关心镜像仓库的更新、升级、维护等。
- 成本低：企业或开发者不需要购买硬件、解决方案来搭建镜像仓库，也不需要团队来维护。

私有镜像仓库优势

- 隐私性：企业的代码和数据是企业的私有资产，不允许随意共享到公共平台。
- 敏感性：企业的镜像会包含一些敏感信息，如密钥信息、令牌信息等。这些敏感信息严禁暴露到企业外部。
- 网络连通性：企业的网络结构多种多样，并非所有环境都可以访问互联网。
- 安全性：而在企业环境中，若使用一些含有漏洞的依赖包，则会引入安全隐患。

Harbor

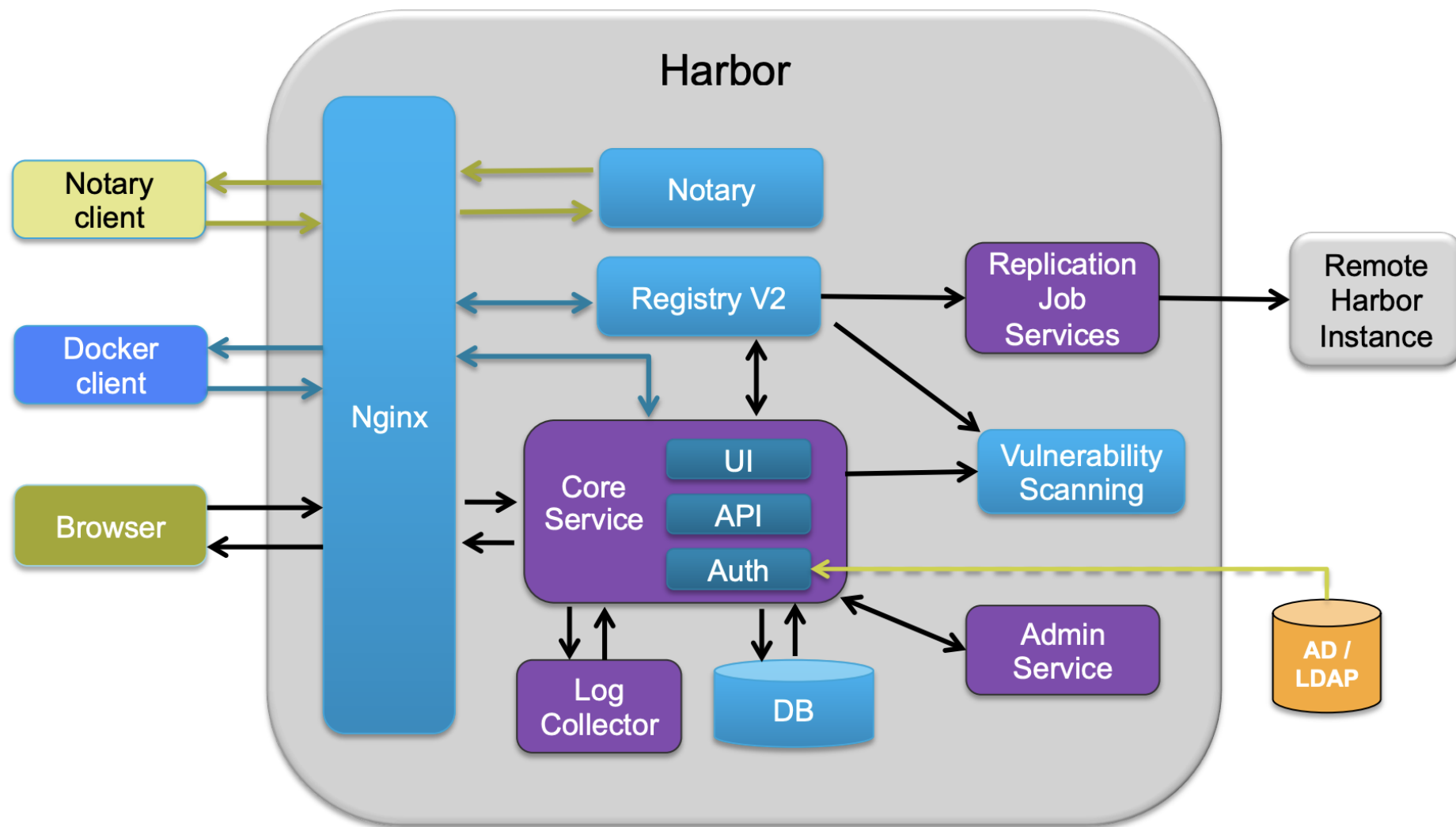
Harbor是VMware开源的企业级镜像仓库，目前已是CNCf的毕业项目。它拥有完整的仓库管理、镜像管理、基于角色的权限控制、镜像安全扫描集成、镜像签名等。



Harbor提供的服务

- Harbor核心服务：提供Harbor的核心管理服务API，包括仓库管理、认证管理、授权管理、配置管理、项目管理、配额管理、签名管理、副本管理等。
- Harbor Portal：Harbor的Web界面。
- Registry：Registry负责接收客户端的pull/push请求，其核心为docker/Distribution。
- 副本控制器：Harbor可以以主从模式来部署镜像仓库，副本控制器将镜像从主镜像服务分发到从镜像服务。
- 日志收集器：收集各模块的日志。
- 垃圾回收控制器：回收日常操作中删除镜像记录后遗留在块存储中的孤立块文件。

Harbor架构



Harbor安装

helm repo add harbor <https://helm.goharbor.io>

helm fetch harbor/harbor --untar

k create ns harbor

helm install -n harbor harbor .

Harbor Demo Server

- Demo Server

2天清理一次数据

不能push超过100Mb的镜像

不能使用管理功能

- Portal

<https://demo.goharbor.io/harbor/projects>

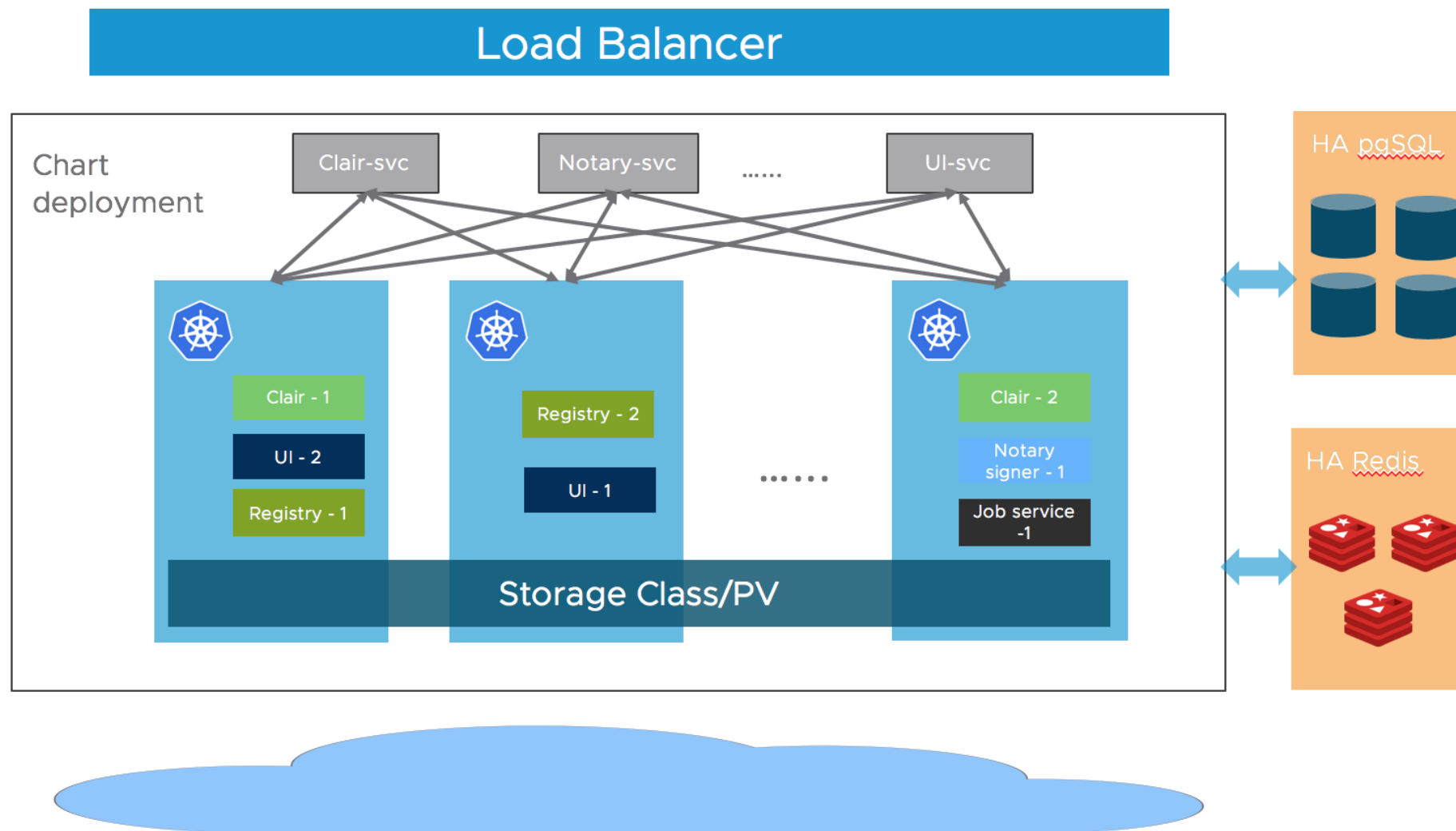
- 镜像使用

`docker login demo.goharbor.io`

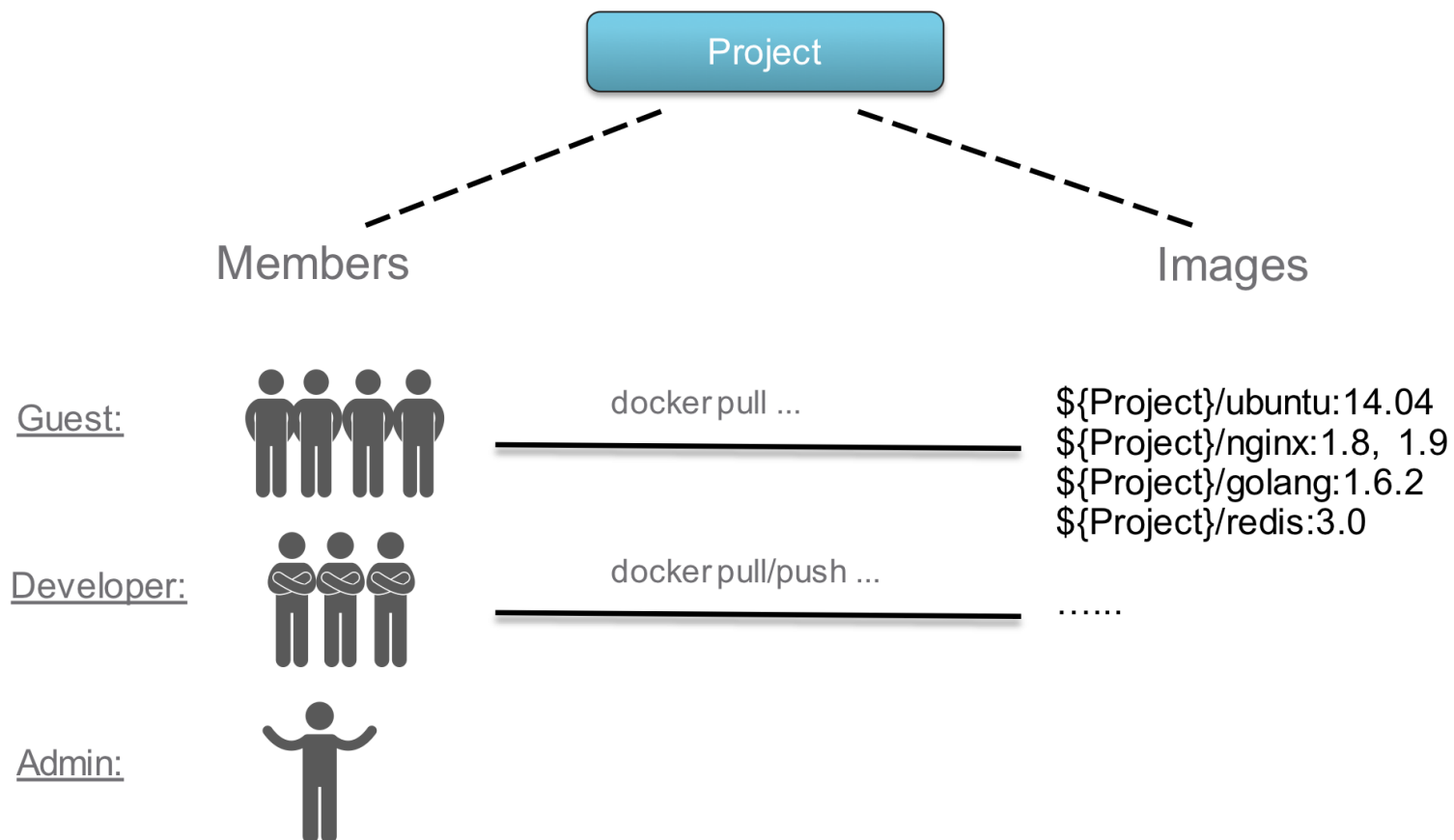
`docker build -t demo.goharbor.io/your-project/test-image .`

`docker push demo.goharbor.io/your-project/test-image`

Harbor高可用架构



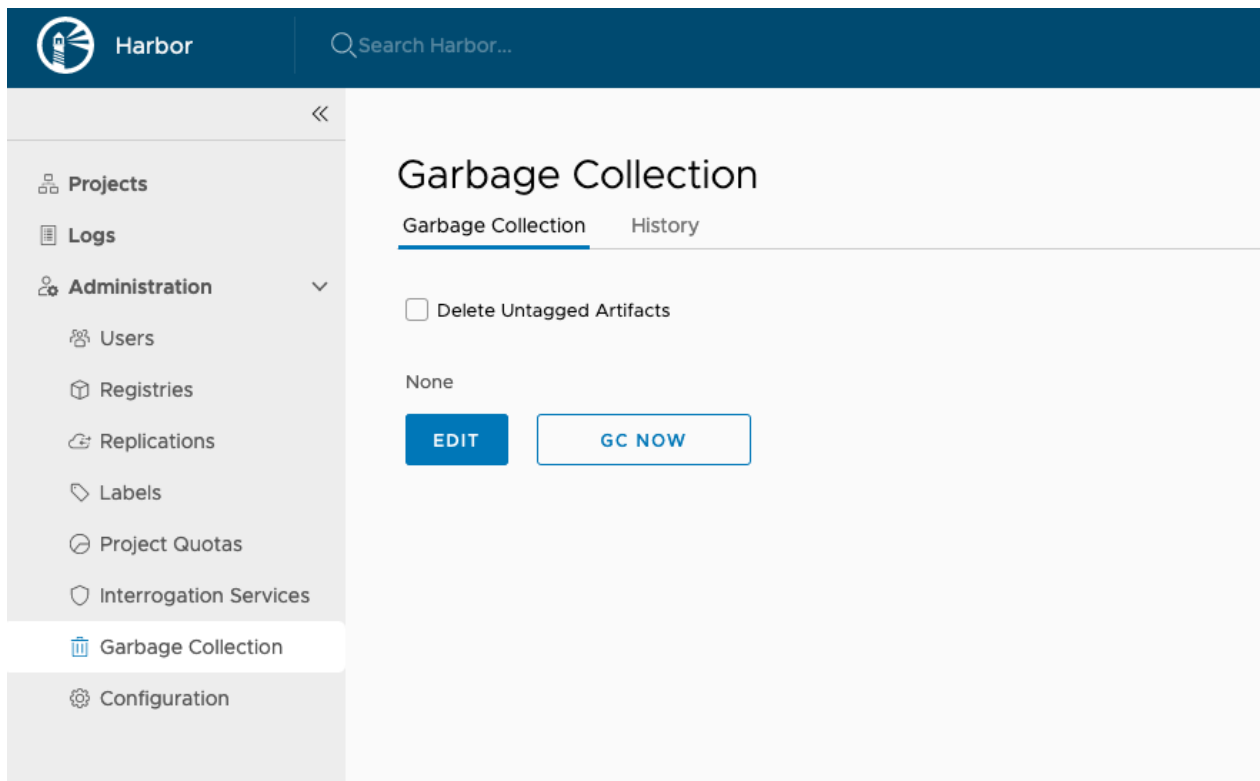
Harbor的用户管理



垃圾回收

镜像删除时，blob文件不会被删除。（你能想起来为什么吗）

需要通过垃圾回收机制来删除不用的blob，进而回收存储空间。



本地镜像加速Dragonfly

Dragonfly 是一款基于 P2P 的智能镜像和文件分发工具

它旨在提高文件传输的效率和速率，最大限度地利用网络带宽，尤其是在分发大量数据时

- 应用分发
- 缓存分发
- 日志分发
- 镜像分发

优势

基于 P2P 的文件分发

非侵入式支持所有类型的容器技术

机器级别的限速

被动式 CDN

高度一致性

磁盘保护和高效 IO

高性能

自动隔离异常

对文件源无压力

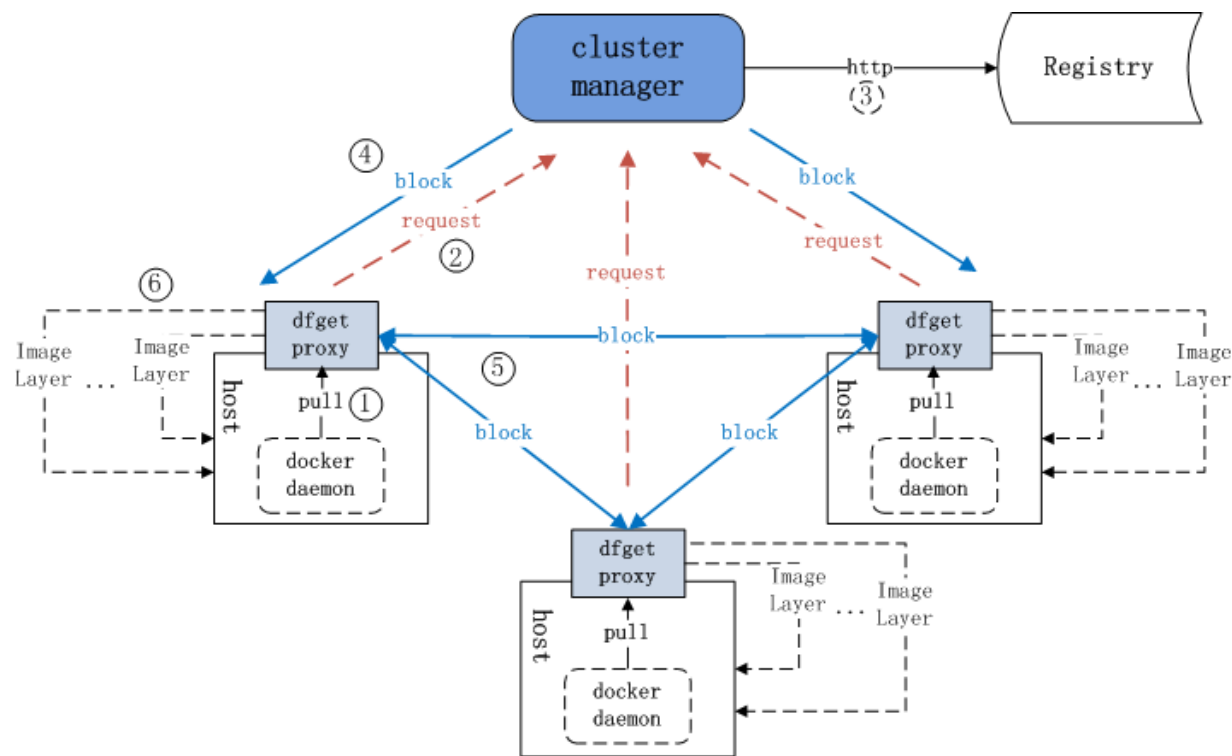
支持标准 HTTP 头文件

有效的 Registry 鉴权并发控制

简单易用

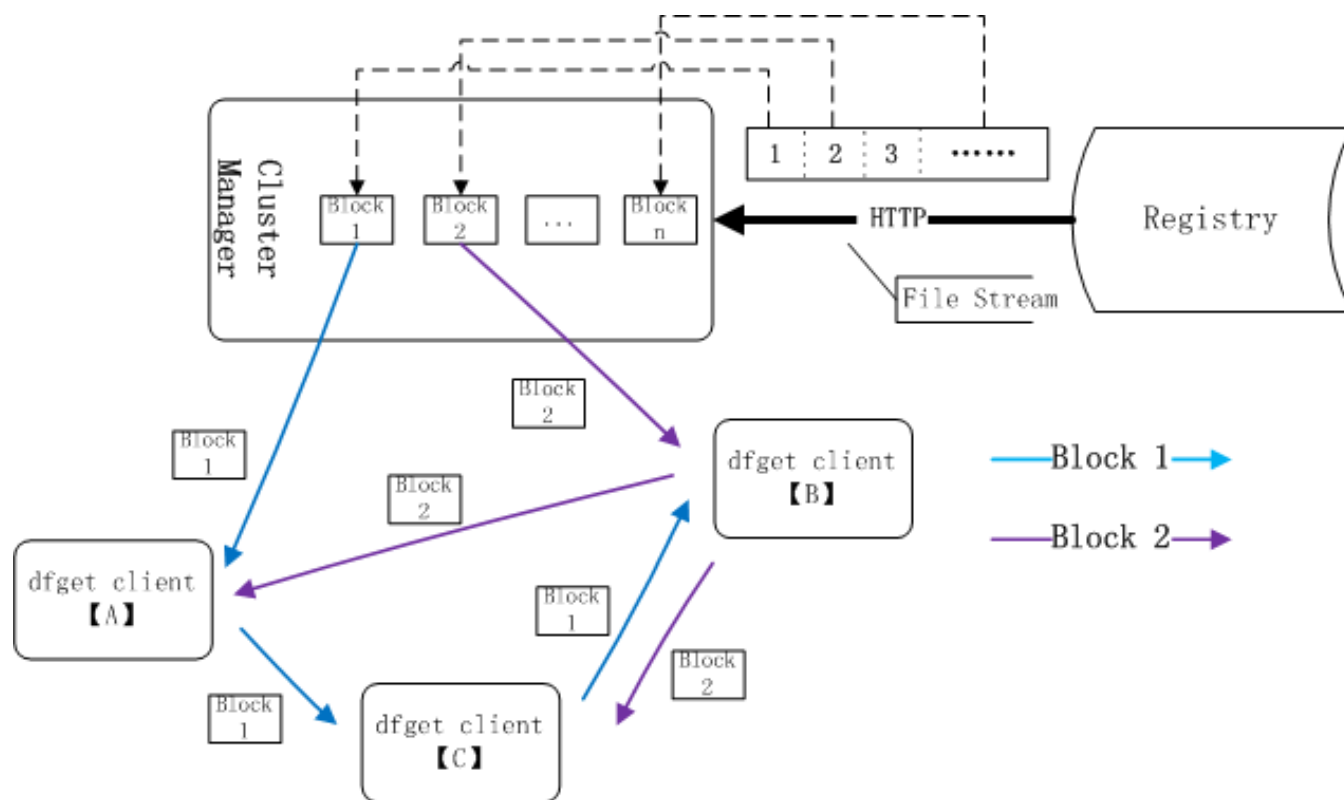
镜像下载流程

dfget proxy 也称为 dfdaemon，会拦截来自 docker pull 或 docker push 的 HTTP 请求，然后使用 dfget 来处理那些跟镜像分层相关的请求



镜像下载流程

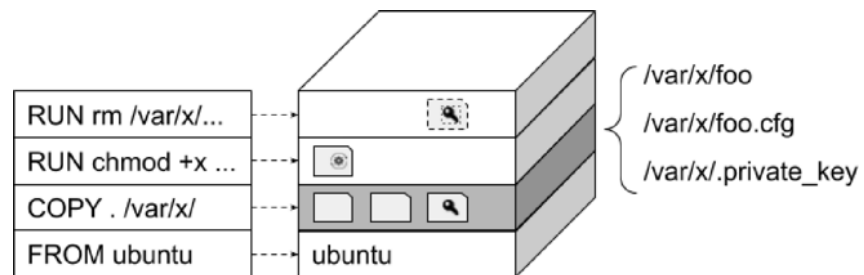
每个文件会被分成多个分块，并在对等节点之间传输



2. 镜像安全

镜像安全的最佳实践

- 构建指令问题
 - 避免在构建镜像时，添加密钥，Token等敏感信息（配置与代码应分离）
- 应用依赖问题
 - 应尽量避免安装不必要的依赖
 - 确保依赖无安全风险，一些长时间不更新的基础镜像的可能面临安全风险，比如基于openssl1.0，只支持tls1.0等
- 文件问题
 - 在构建镜像时，除应用本身外，还会添加应用需要的配置文件、模板等，在添加这些文件时，会无意间添加一些包含敏感信息或不符合安全策略的文件到镜像中。
 - 当镜像中存在文件问题时，需要通过引入该文件的构建指令行进行修复，而不是通过追加一条删除指令来修复。



镜像扫描 (Vulnerability Scanning)

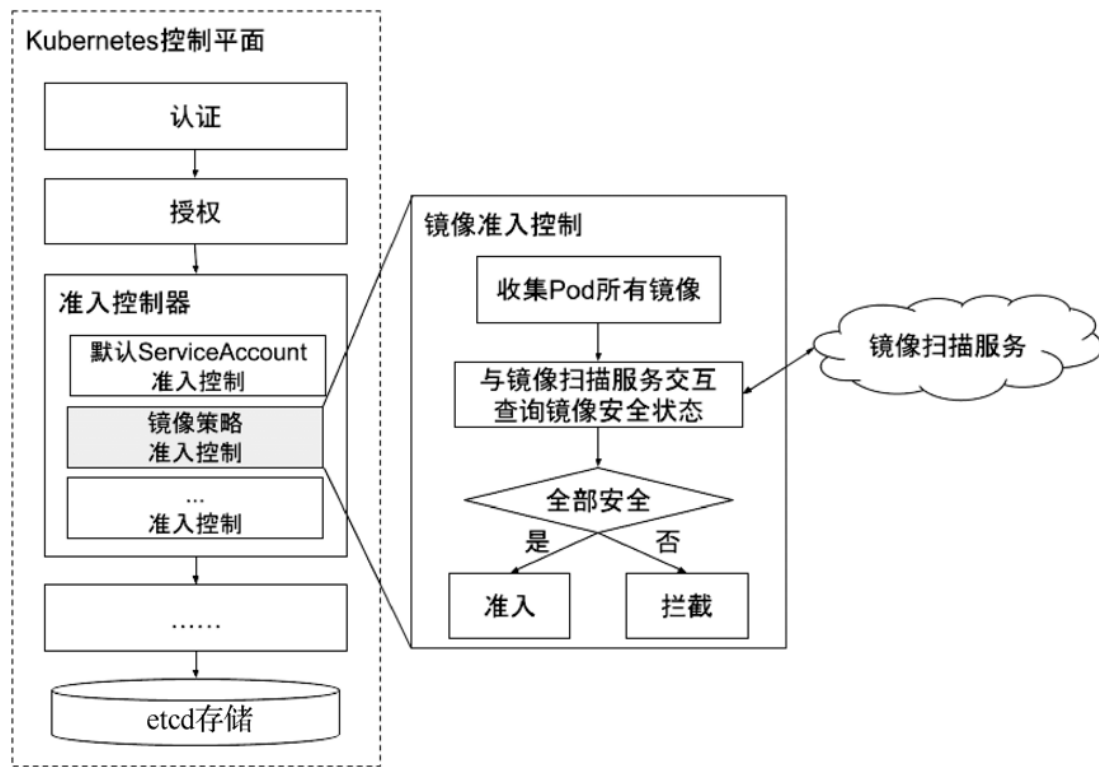
镜像扫描通过扫描工具或扫描服务对镜像进行扫描，来确定镜像是否安全

- 分析构建指令、应用、文件、依赖包
- 查询CVE库、安全策略
- 检测镜像是否安全，是否符合企业的安全标准。

镜像策略准入控制

镜像准入控制是在部署Pod、更新Pod时，对Pod中的所有镜像进行安全验证以放行或拦截对Pod的操作：

- 放行：Pod中所有的镜像都安全，允许此次的操作，Pod成功被创建或更新。
- 拦截：Pod中的镜像未扫描，或已经扫描但存在安全漏洞，或不符合安全策略，Pod无法被创建或更新。



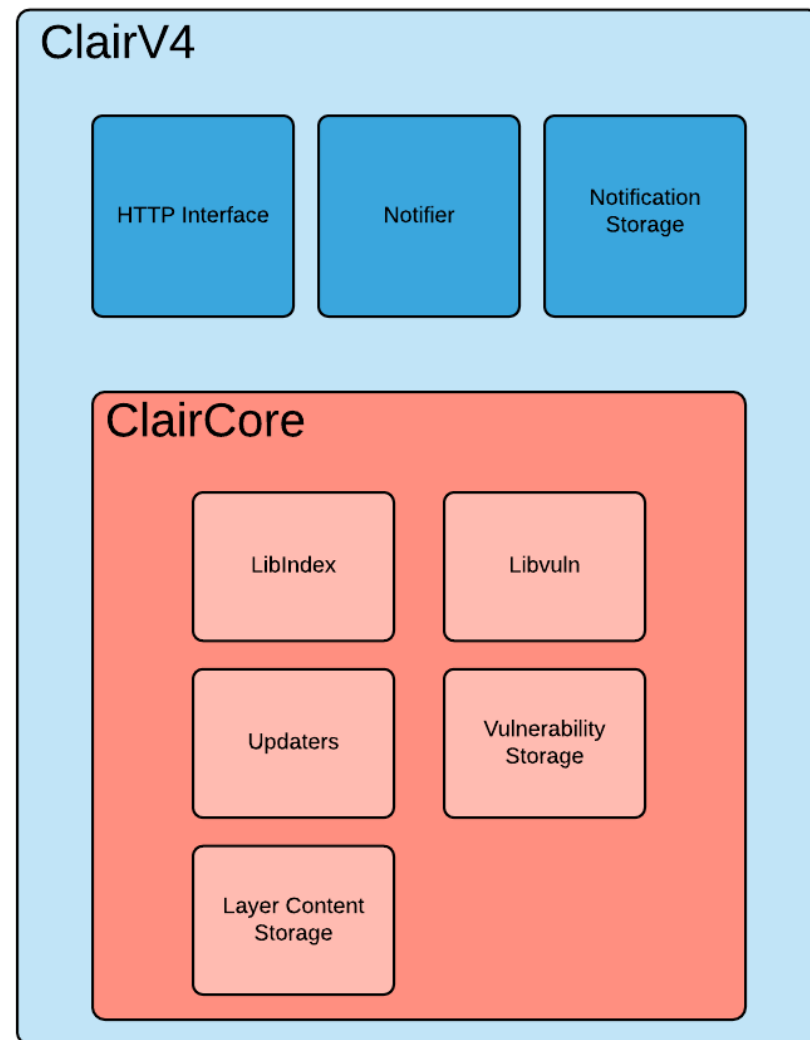
扫描镜像

1. 镜像扫描服务从镜像仓库拉取镜像。
2. 解析镜像的元数据。
3. 解压镜像的每一个文件层。
4. 提取每一层所包含的依赖包、可运行程序、文件列表、文件内容扫描。
5. 将扫描结果与CVE字典、安全策略字典进行匹配，以确认最终镜像是否安全。

镜像扫描服务

供应商	Anchore	Aqua	Twistlock	Clair	Qualys
镜像文件扫描	支持	支持	支持	支持	支持
多CVE库支持	支持	支持	支持	支持	支持
是否开源	是	部分		是	
商业支持	支持	支持	支持		支持
可定制安全策略	支持		支持		
镜像仓库支持	Harbor			Quay、Harbor	

Clair架构



*课后练习10.1

将Nginx容器镜像上传至Harbor Demo server并运行在测试环境中。

3. 基于Kubernetes的DevOps

传统运维模式

- 缺乏一致性环境
- 平台与应用部署相互割裂
- 缺乏工具链支持
- 缺乏统一的灰度发布管理
- 缺乏统一监控能力和持续运维能力

建立持续交付的服务体系

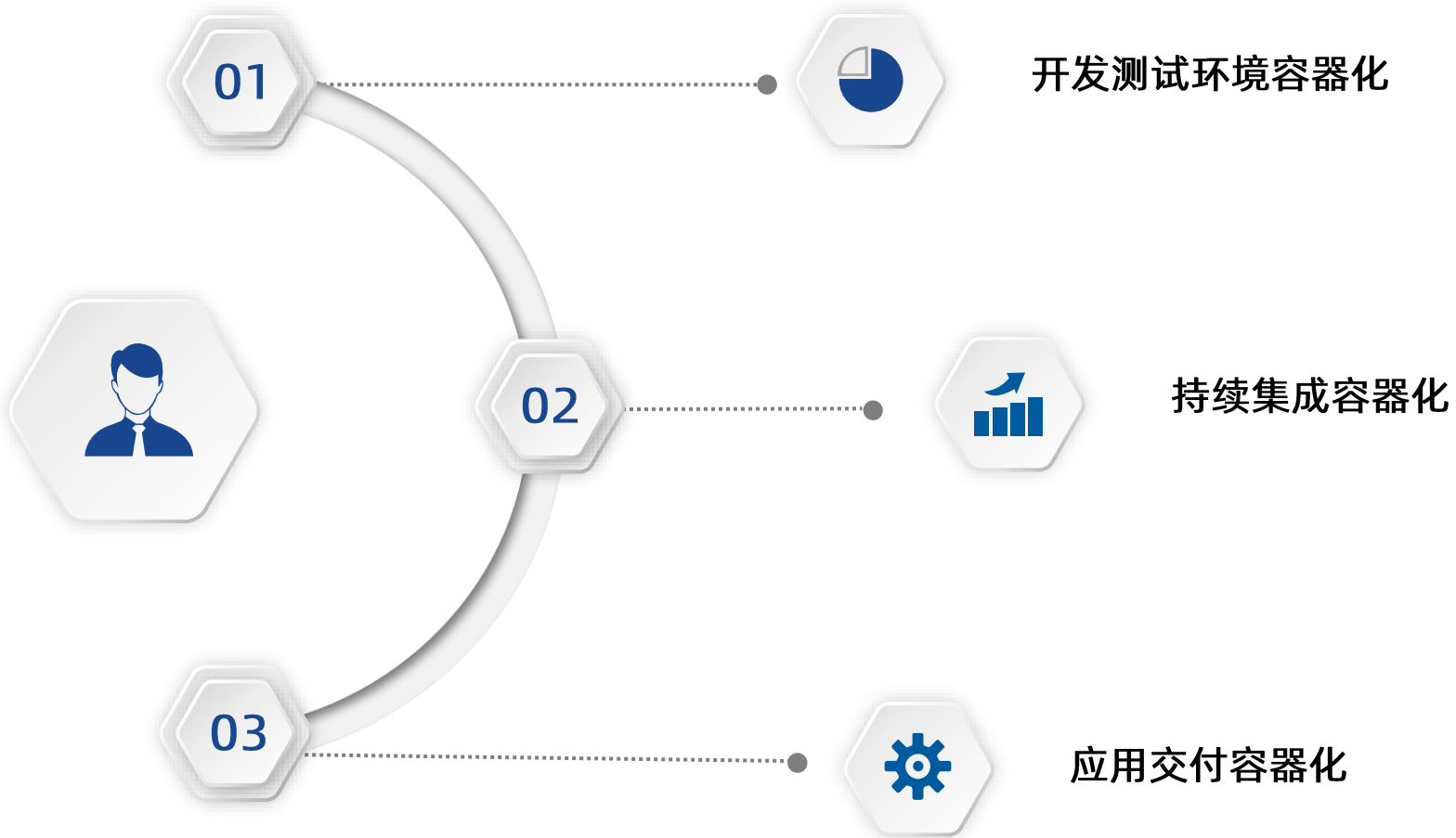
传统的开发运维模式下，存在的问题：

- 从需求到版本上线中间是个黑箱子，风险不可控；
- 开发设计时未过多考虑运维，导致后续部署及维护的困难；
- 开发各自为政，烟囱式开发，未考虑共享重用、联调，开发的资产积累不能快速转移到运维手中；

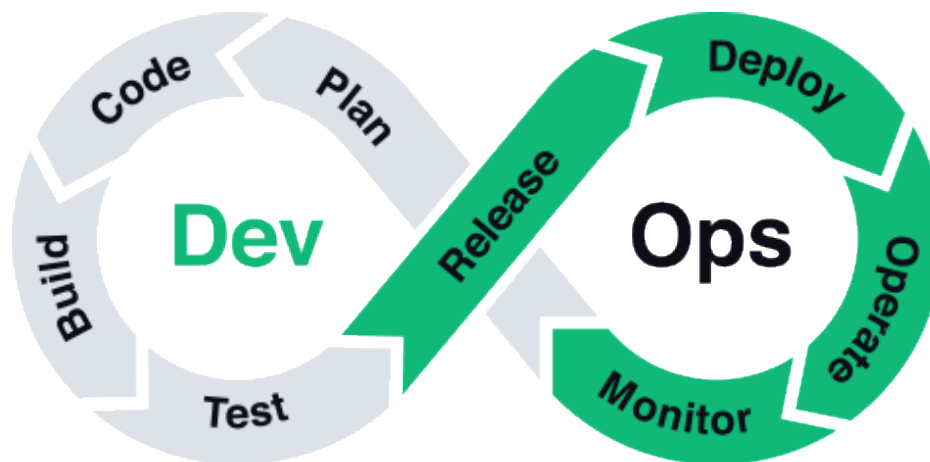
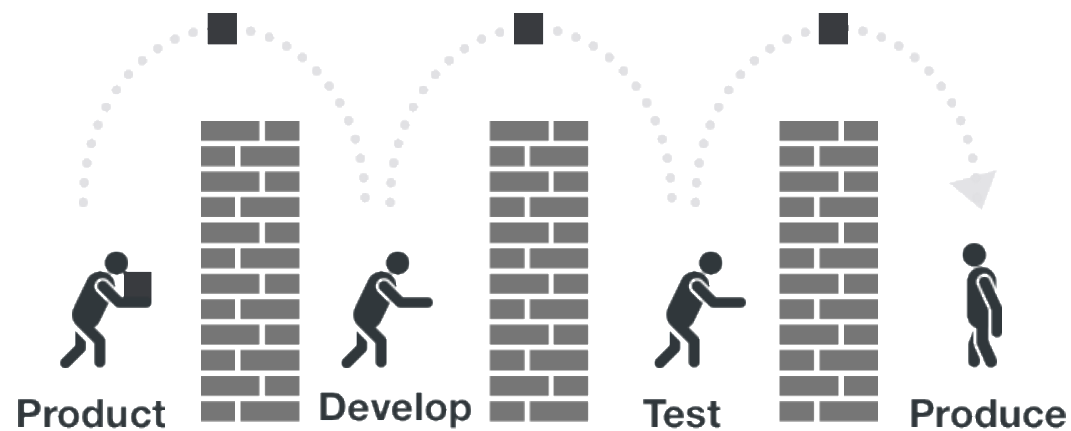


应对这样的问题，我们通常倡导的解决之道是：运维前移，统一运维，建立持续交付服务体系。

基于Docker的开发模式驱动持续集成



DevOps流程定义



Dev和Ops的边界定义

Programming vs. Engineering

- Programming更多的是系统设计和编码实现
- Engineering包含更大范围概念，除了功能层面的实现，还需为运维服务

定义production readiness

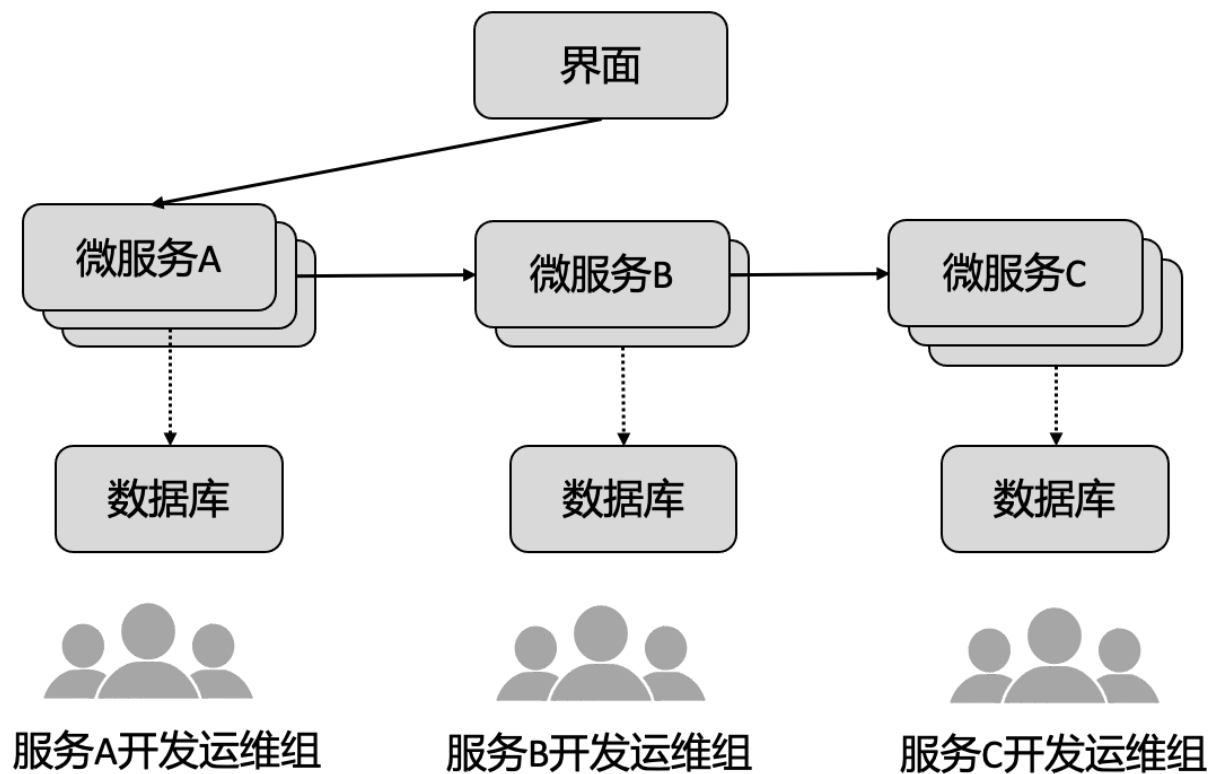
Function ready vs. production ready

- Function Ready只是交付的软件产品从功能层面满足需求定义
- Production Ready除了功能就绪还包含
 - LnP测试通过，满足性能需求
 - 用户手册完成，用户可按照用户手册使用既定功能
 - 管理手册完成，运维人员可以依照管理手册部署，升级产品并解决现网问题
 - 监控，包括
 - 组件健康状态检查（UP）
 - 性能指标（Metrics）
 - 基于性能指标，定义alert rule，在系统故障或缓慢时，发送告警信息给运维人员
 - Assertion，定期测试某功能并检查结果，比如每小时创建service，测试vip连通性

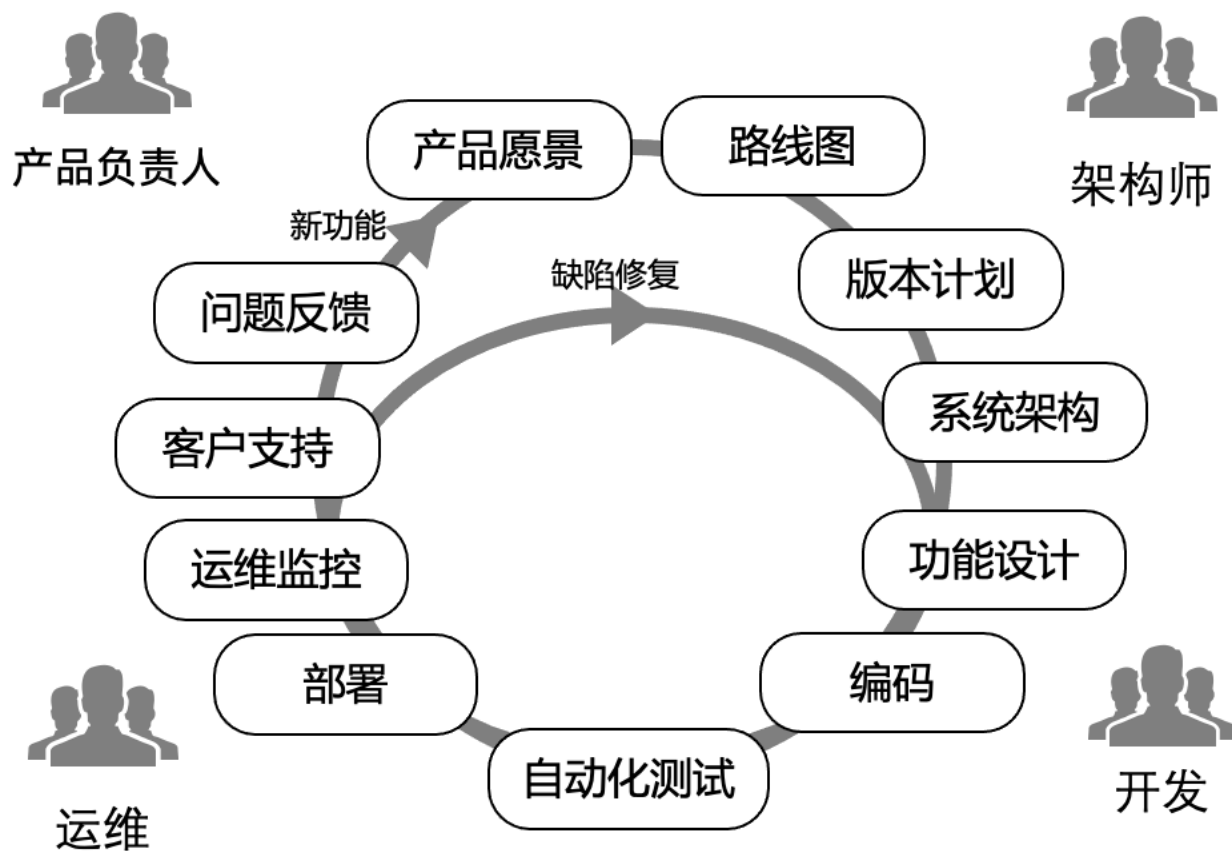
单体架构下的人员配置



微服务架构下的人员配置



DevOps下的人员划分



此组织结构的优缺点

优势

- 一个架构师负责整个产品的规划，使得产品更规范，产品进化不同的Program由不同PO负责端到端，从需求到生产系统部署，保证solution质量
- 研发和运营统一，一个最大的作用是研发可以深刻理解现网痛苦，对功能需求的设计，和优先级定义有极大帮助。
- Function ready vs. production ready
- Programming vs. Engineering
- 更具连续性

- 传统运营和开发的冲突并非消失，而是转变为了运营经理和架构以及PO之间的冲突。
- 运营轮值导致生产系统权责不明确
- 轮值期间期待不出故障
- 对于出现的故障缺少端对端的跟踪，可能问题还没处理结束，已经要交接给下一个人了

问题

- 对于生产系统的积累问题，如故障节点，无人专职处理，导致比较多的故障节点堆积
- 对于生产系统的配置，无统一规划，每个人可能用不同的配置达到相同的目的
- Dev自主权过高导致Dev可以偷偷加功能并部署到生产

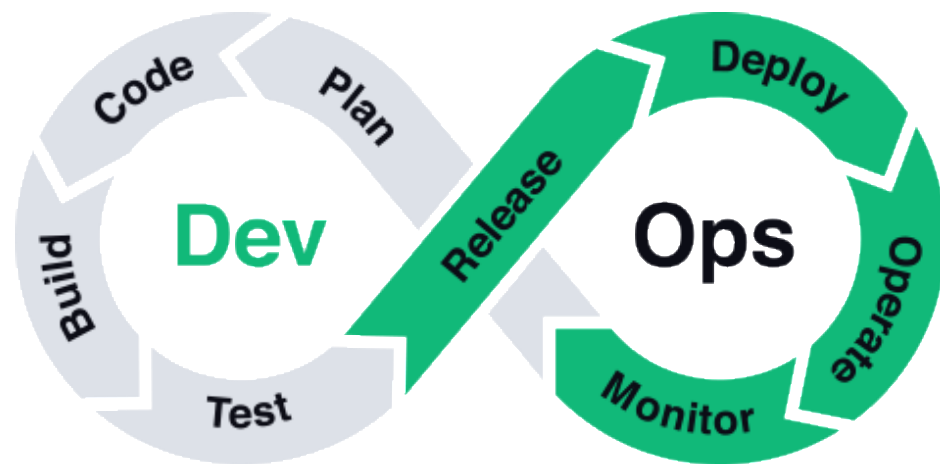


我眼中理想的DevOps

Dev和Ops需要责权划分，可以有overlap，同时做部署，同时做计划，但Dev应侧重功能开发，Ops偏重生产系统运维

Ops参与到版本规划流程中，并为一个功能能不能release和deploy把关

- Dev
 - Plan
 - Code
 - Build
 - Test
 - Release
 - Deploy
- Ops
 - Deploy
 - Operate
 - Monitor
 - Plan



产品愿景

产品愿景的定义有多个目的：

- 统一团队思想，让团队成员知道我们要往哪里去，这有助于让团队专注于交付产品价值。
- 长期愿景往往是有野心的，能对团队成员起到激励作用，比如“业界尖端技术”往往能刺激团队成员努力自我提升。
- 该愿景应该是团队成员的共识，是所有人的共同理想。
- 愿景要产生真正的价值，是真正建立在对当前业务痛点的充分理解基础之上的。

长期愿景

基于业界尖端技术打造下一代流量管理平台

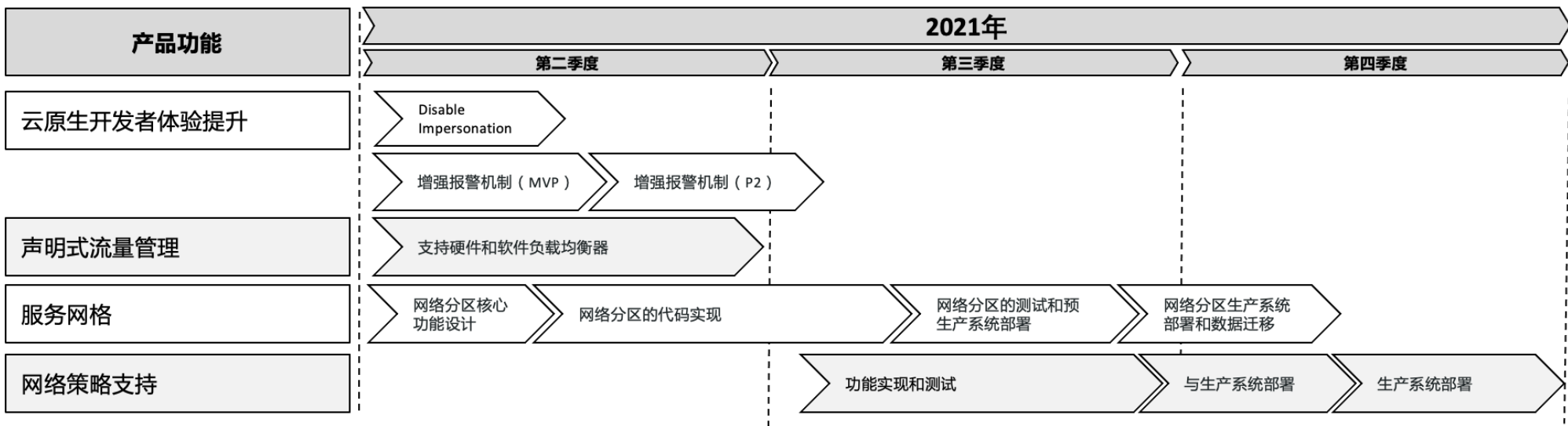
产品价值



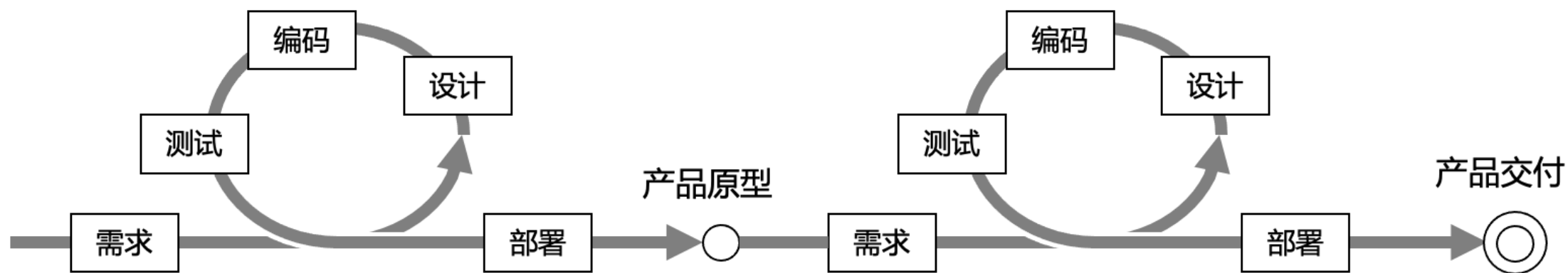
- 节省时间成本，负载均衡上架时间从数月降低到分钟级
- 移除供应商依赖问题，出现生产系统故障不再依赖供应商上门调试
- 构建一套统一模型管理所有业务场景，降低系统集成成本
- 全自动化，减少手工操作，降低维护人力成本
- 故障检测和根因分析能力，快速定位故障，提升可用性

产品路线图定义

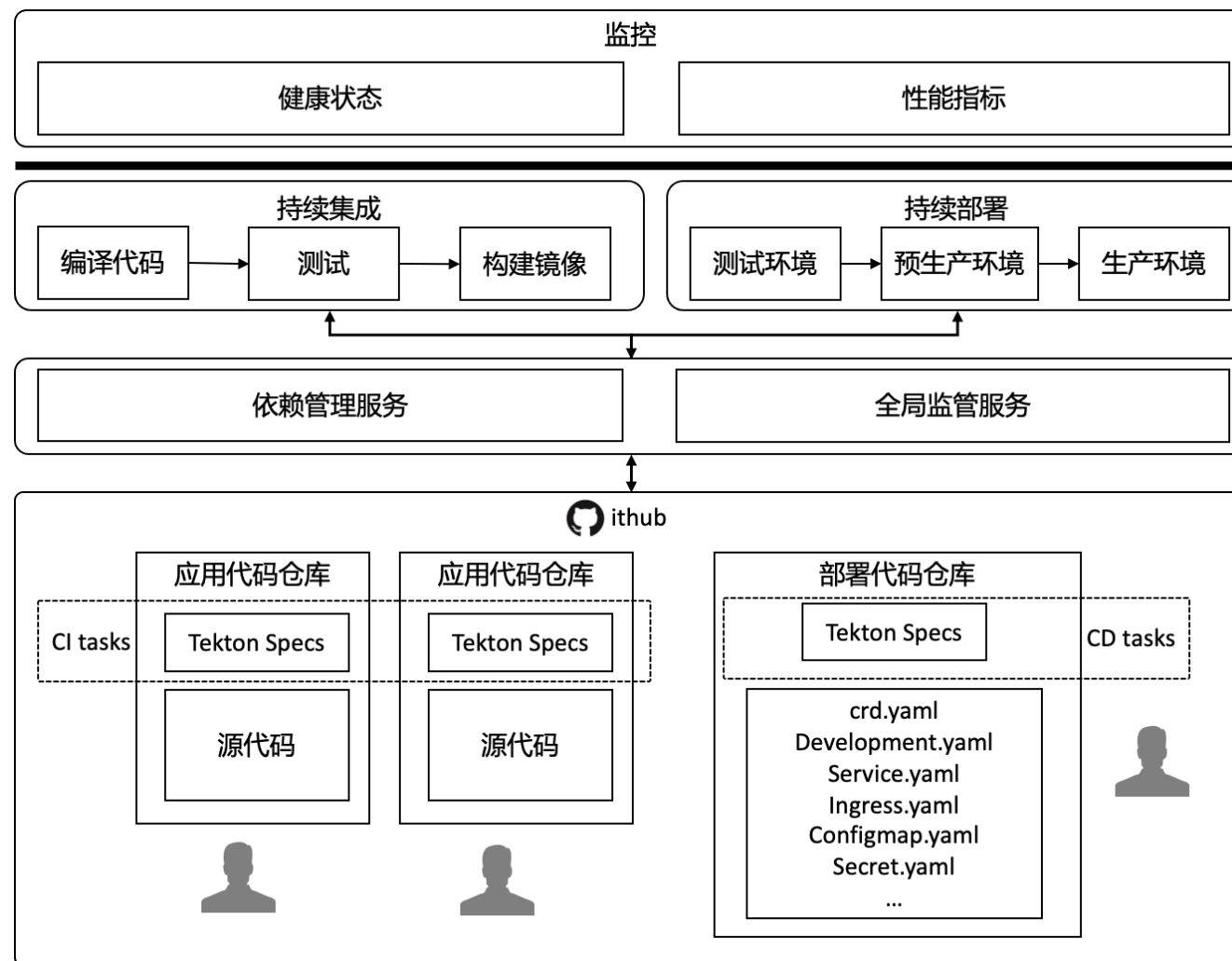
而项目执行需要有明确时间线的近期目标，因此需要将产品价值转化成可控的产品需求。产品需求的输入包括新功能和当前产品的功能缺陷等，产品经理需要与核心团队一道定义近期（比如2-3个季度）的产品核心功能，并定义近期产品版本需要包含的功能。



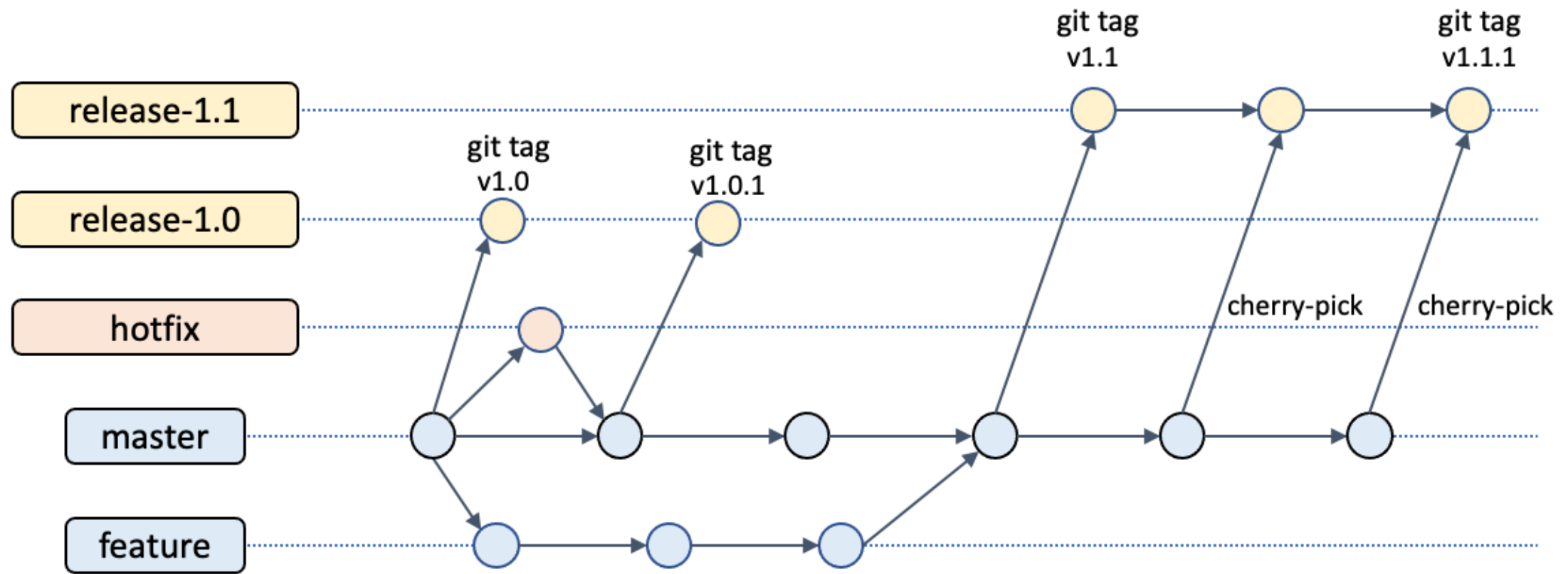
敏捷开发



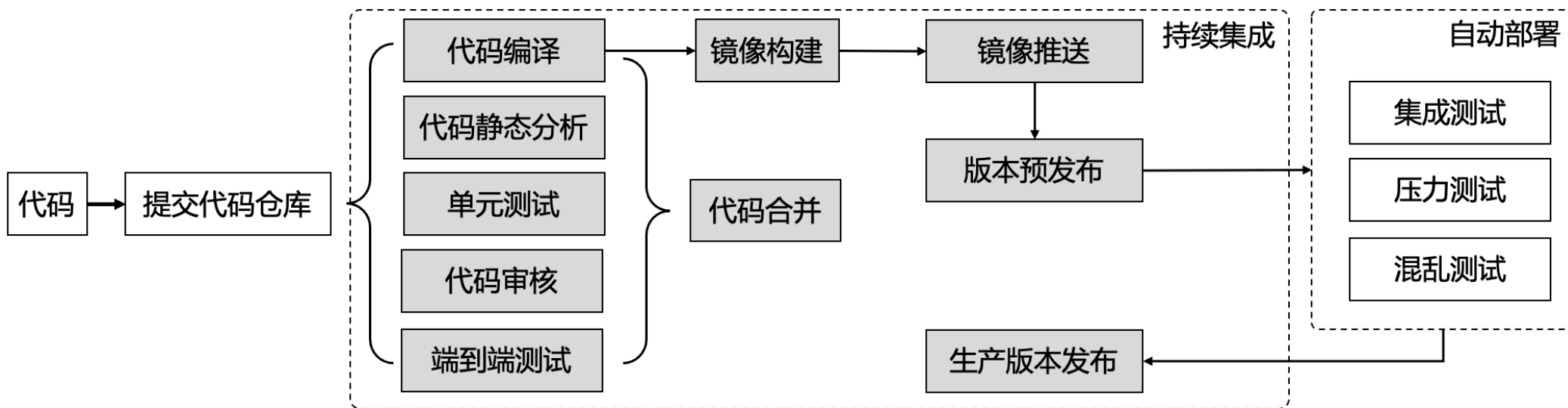
DevOps流程概览



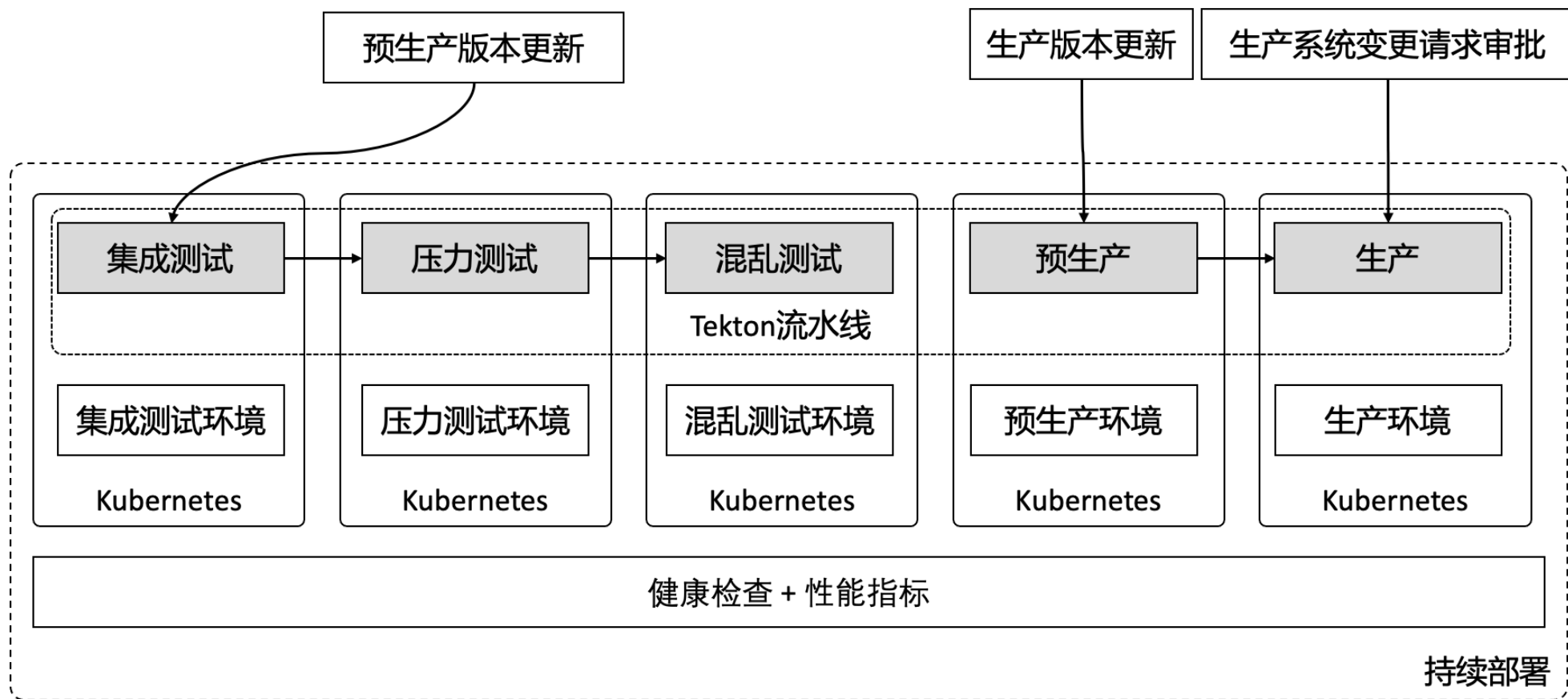
代码分支管理



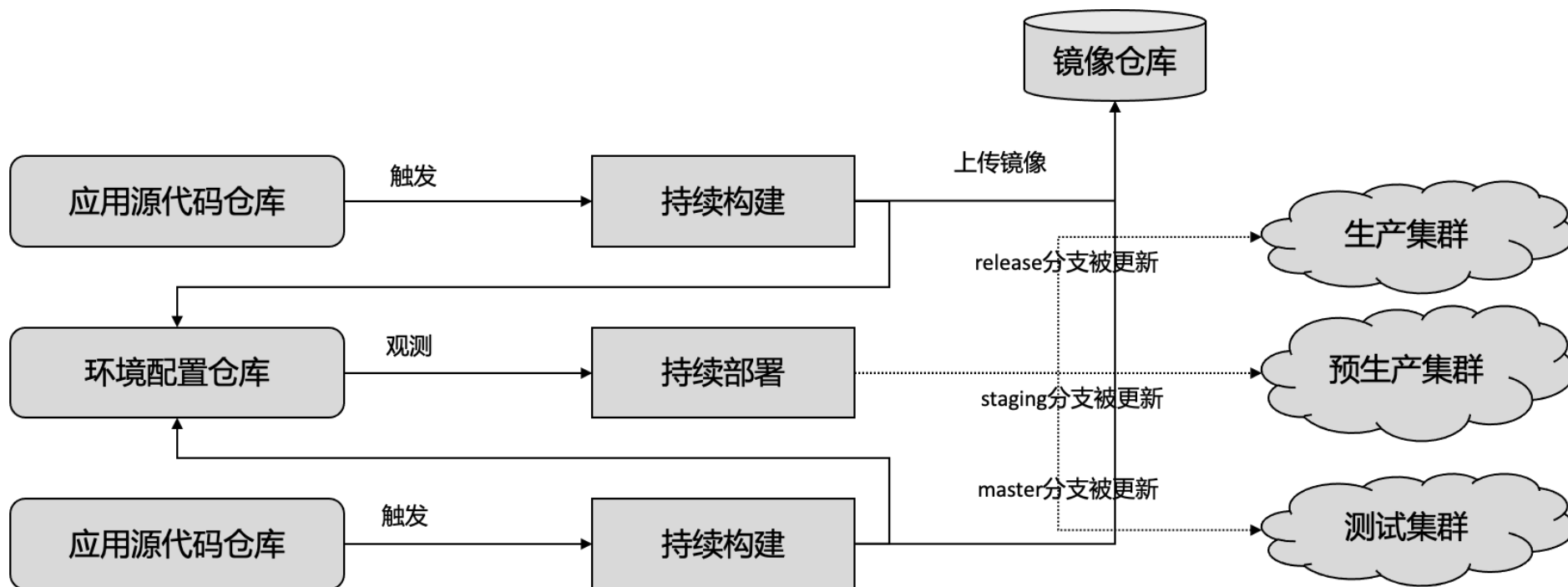
持续集成



持续部署



GitOps



4. 基于GitHub Action的自动化流水线

基于公共GitHub的action构建流水线

- 低成本
 - GitHub目前为项目提供免费构建流水线，可满足日常构建需求
- 免运维
 - 无需自己构建流水线，GitHub提供小量构建请求
- 易构建
 - GitHub action非常容易构建，只需点击几次按钮，即可完成
 - GitHub 提供了一系列内建action，社区有大量可复用的action
- 易集成
 - 无需配置GitHub Webhook即可完成与PR的联动

Action的创建

[Pull requests](#) 1 [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

Choose a workflow template

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow template to get started.

Skip this and [set up a workflow yourself](#) →

Workflows made for your repository Suggested

Docker image

By GitHub Actions



Build a Docker image to deploy, run, or push to a registry.

[Set up this workflow](#)

```
docker build . --file Dockerfile --tag my-image-name:${date +%s}
```

 actions/starter-workflows

Dockerfile ●

Go

By GitHub Actions



Build a Go project.

[Set up this workflow](#)

```
go build -v ./...
go test -v ./...
```

 actions/starter-workflows

Go ●

Build projects with Make

By GitHub Actions



Build and test a project using Make.

[Set up this workflow](#)

```
./configure
make
make check
```

 actions/starter-workflows

Makefile ●

Publish Docker Container

By GitHub Actions



Build, test and push Docker image to GitHub Packages.

[Set up this workflow](#)

 actions/starter-workflows

Dockerfile ●

Action细节

<https://github.com/cncamp/golang/blob/master/.github/workflows/go.yml>

on:

pull_request:

branches: [master]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v2

- name: Set up Go

uses: actions/setup-go@v2

with:

go-version: 1.17

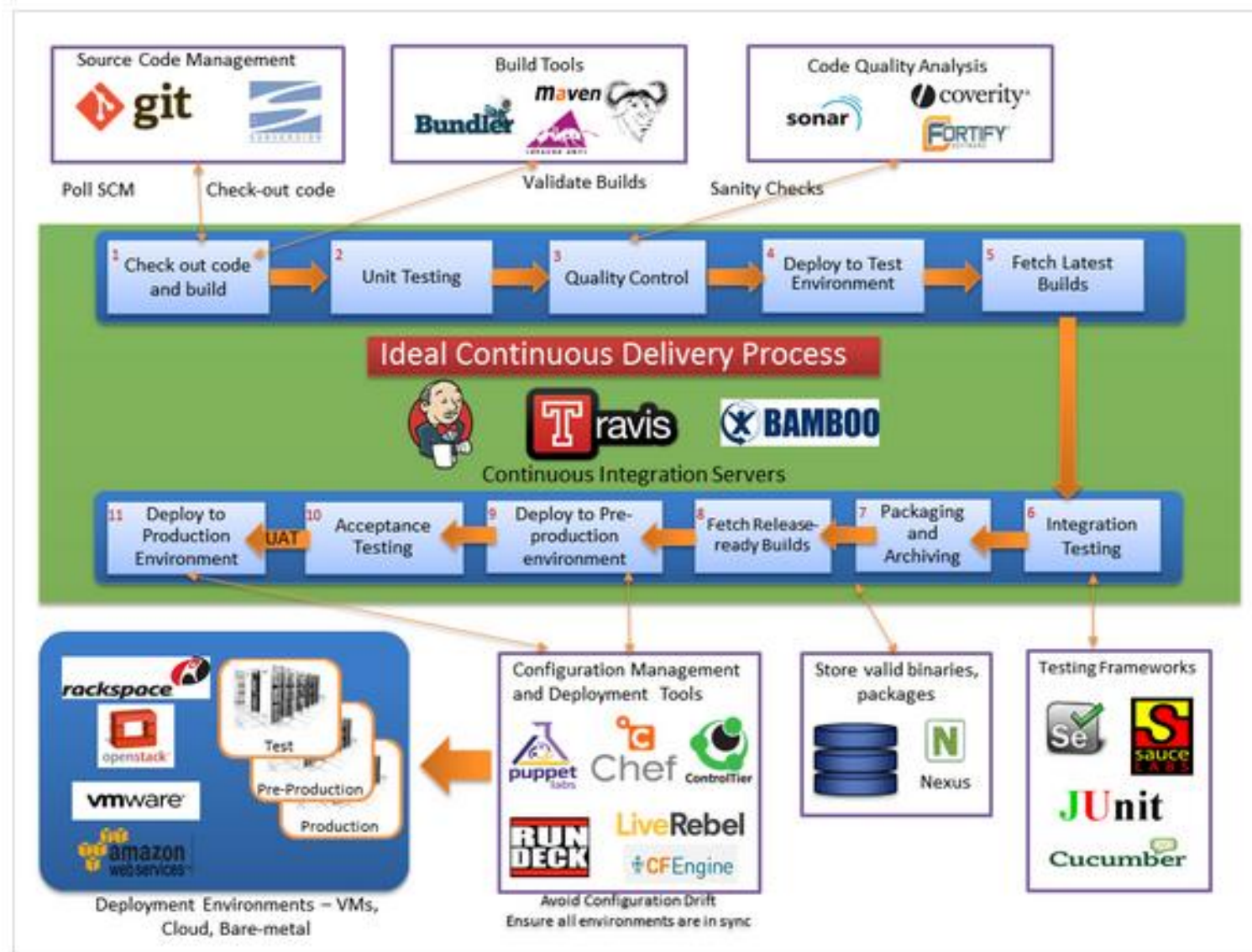
- name: Build

run: go build -v ./...

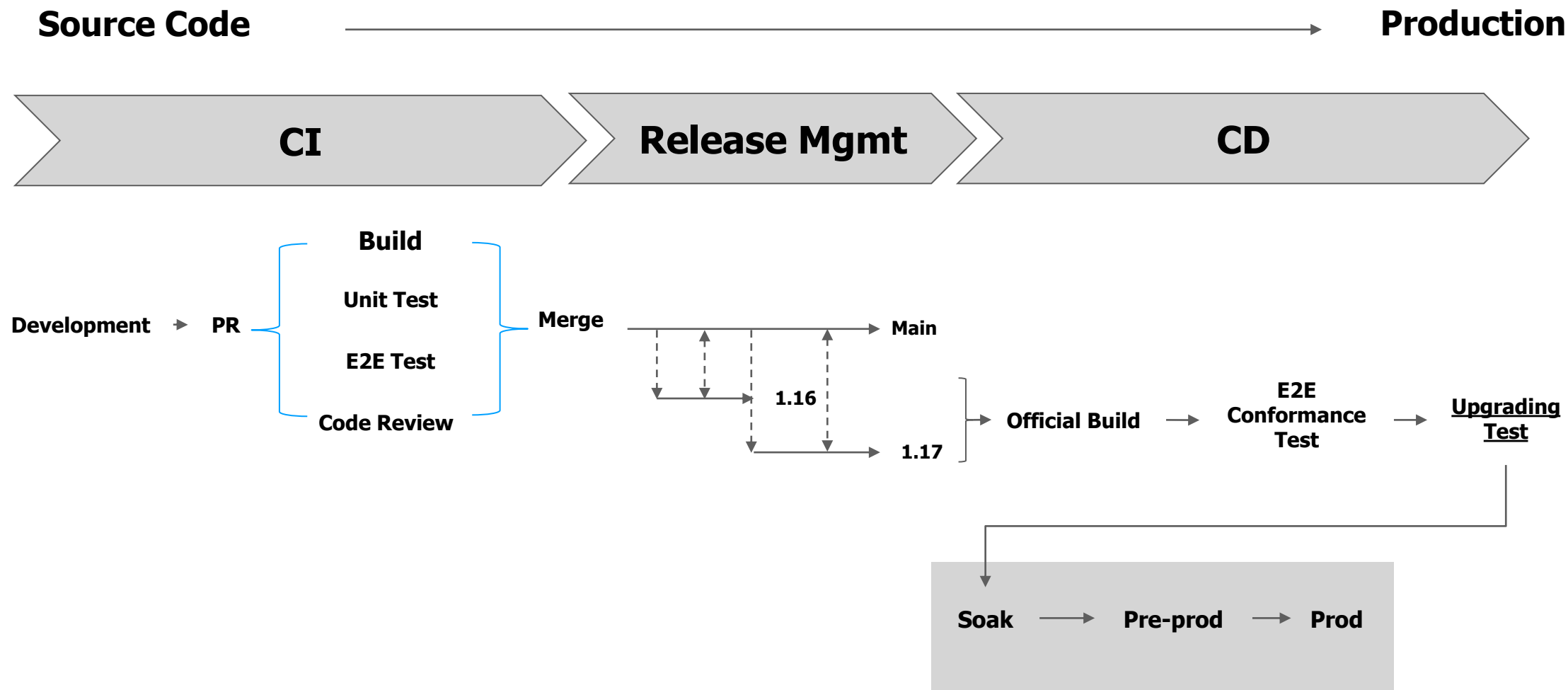
- name: Test

run: go test -v ./...

5. 基于Jenkins的自动化流水线



Kubernetes CI&CD 完整流程

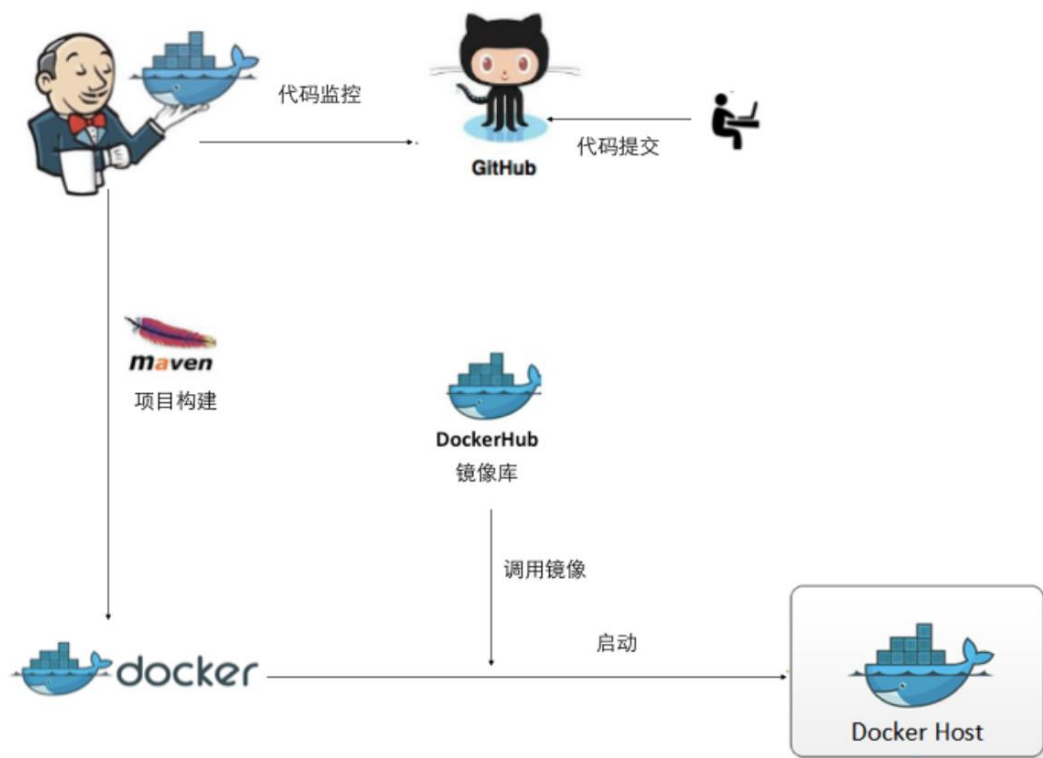


持续集成容器化

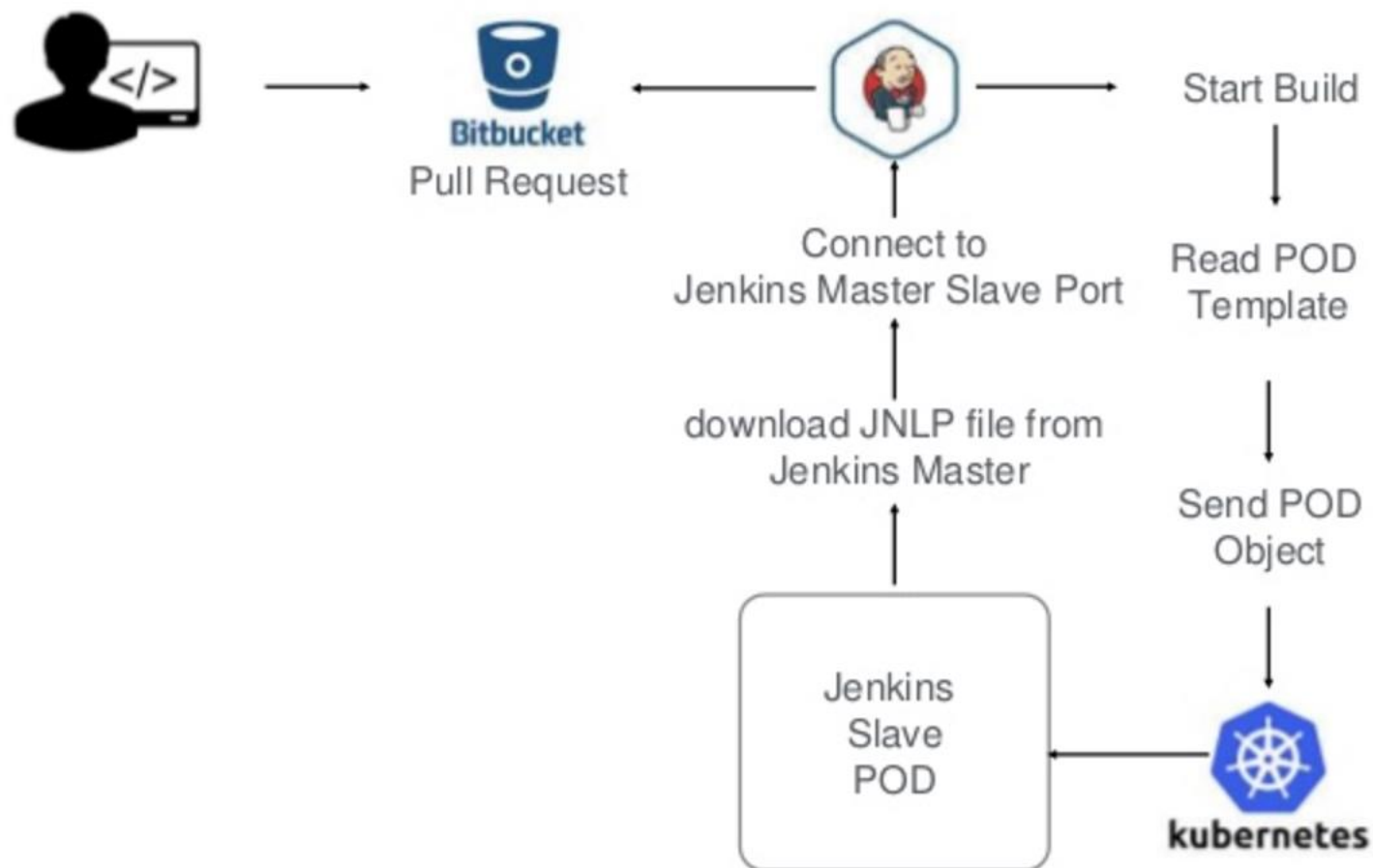
最主要解决的问题是保证CI构建环境和开发构建环境的统一。

使用容器作为标准的构建环境，将代码库作为Volume挂载进构建容器。

由于构建结果是Docker镜像，所以要在构建容器中执行“docker build”命令，需要注意DIND的问题。



基于Kubernetes的持续集成



Docker in Docker问题展开

- 方法1: docker in docker

早期尝试

<https://github.com/jpetazzo/dind> (achived)

官方支持

https://hub.docker.com/_/docker/

```
docker run --privileged -d docker:dind
```

可能引入的问题:

<https://jpetazzo.github.io/2015/09/03/do-not-use-docker-in-docker-for-ci/>

if your use case *really absolutely* mandates Docker-in-Docker, have a look at [sysbox](#), it might be what you need.

Docker in Docker问题展开

方法2: mount host docker.socket

```
docker run -v /var/run/docker.sock:/var/run/docker.sock ...
```

思考: 可能引入的问题?

Docker in Docker问题展开

- 方法3: Kaniko

<https://github.com/GoogleContainerTools/kaniko>

```
echo -e 'FROM alpine \nRUN echo "created from standard input"' > Dockerfile | tar -cf -  
Dockerfile | gzip -9 | docker run \  
  --interactive -v $(pwd):/workspace gcr.io/kaniko-project/executor:latest \  
  --context tar://stdin \  
  --destination=cncamp/echoimage
```

构建基于Kubernetes的Jenkins Pipeline

1. Image准备：基于Jenkins 官方Image安装自定义插件

a. <https://github.com/jenkinsci/kubernetes-plugin>

i. 默认安装kubernet plugin

b. <https://github.com/jenkinsci/docker-jnlp-slave>

i. 如果需要做docker build, 则需要自定义Dockerfile, 安装docker binary, 并且把host的 docker.socket mount进container

2. Jenkins配置的保存

a. 需要创建PVC, 以便保证在jenkins master pod出错时, 新的kubernetes pod可以mount同样的工作目录, 保证配置不丢失

b. 可以通过jenkins scm plugin把jenkins配置保存至GitHub

3. 创建Kubernetes spec

Jenkins的配置

Cloud provider配置

- 在Jenkins System Config选择点击Cloud，并选择kubernetes
- 创建Kubernetes ServiceAccount，并在kubernetes 对应的namespace授予namespace admin权限
- 指定Jenkins slave的 image
- 按需指定volume mount，如需在slave container内部执行docker 命令，则需要mount /var/run/docker.sock

创建Jenkins Job

- Git Integration
 - Jenkins webhook on Github
 - Git review/merge Bot
- Build job
 - Git clone code
 - Build binary
 - Build Docker image
 - Push Docker image to hub
- Testing
 - UT / UT Coverage
 - E2E
 - Conformance
 - Conformance Slow
 - LnP

Jenkins练习

- 创建Jenkins Master

```
kubectl apply -f Jenkins.yaml
```

```
kubectl apply -f sa.yaml
```

- 等待Jenkins-0 pod运行, 查看日志查找root 密码

```
kubectl logs -f Jenkins-0
```

- 查看Jenkins Service的NodePort, 登录Jenkins console

<http://192.168.34.2:<nodePort>>

- 输入root密码并创建管理员用户, 登录

安装Kubernetes插件

- 菜单Manage Jenkins->Manage Plugins->Available
- 查找并安装Kubernetes
 - 选择Kubernetes, 点击install without restart
- 等待安装完成

配置Cloud Provider

- 菜单Manage Jenkins->Manage Node and Cloud->Configure Cloud
- Add a new cloud: Kubernetes
 - Kubernetes URL: <https://kubernetes.default>
 - Kubernetes Namespace: default
 - Credentials: Add->Jenkins->Kind: Kubernetes Service Account
 - Test connection
 - Jenkins URL: http://jenkins
- PodTemplate
 - Labels: **jnlp-slave**
 - Add Container // 新版本Jenkins会默认用社区镜像启动jnlp slave, 可以通过定义同名容器覆盖
 - Name: **jnlp**
 - Image: cncamp/inbound-agent
 - Command: ""
 - Arguments to pass to the command: \${computer.jnlpmac} \${computer.name}
- save

Create a Job and test

- Dashboard->Create a Job->Freestyle project
- Restrict where this project can be run
 - Label Expression: **jnlp-slave**
- Build->Add build step->Execute shell
 - echo hello world
- Build Now
- 查看jenkins slave pod:
- kubectl get pod

jnlp-hpsqb	1/1	Running	0	5s
------------	-----	---------	---	----

- 查看job log

Building remotely on [jnlp-hpsqb](#) (jnlp-slave) in workspace

```
/home/jenkins/agent/workspace/test [test] $ /bin/sh -xe
```

```
/tmp/jenkins4853086890420857212.sh + echo hello world hello world Finished: SUCCESS
```


6. Tekton

Jenkins的不足

- 基于脚本的Job配置复用率不足
 - Jenkins等工具的流水线作业通常基于大量不可复用的脚本语言，如何提高代码复用率
- 代码调试困难
 - 如何让流水线作业的配置更好地适应云原生场景的需求越来越急迫

基于声明式API的流水线-Tekton

自定义：Tekton对象是高度自定义的，可扩展性极强。平台工程师可预定义可重用模块以详细的模块目录提供，开发人员可在其他项目中直接引用。

可重用：Tekton对象的可重用性强，组件只需一次定义，即可被组织内的任何人在任何流水线都可重用。使得开发人员无需重复造轮子即可构建复杂流水线。

可扩展性：Tekton组件目录（Tekton Catalog）是一个社区驱动的Tekton组件的存储仓库。任何用户可以直接从社区获取成熟的组件并在此之上构建复杂流水线，也就是当你要构建一个流水线时，很可能你需要的所有代码和配置都可以从Tekton Catalog直接拿下来复用，而无需重复开发。

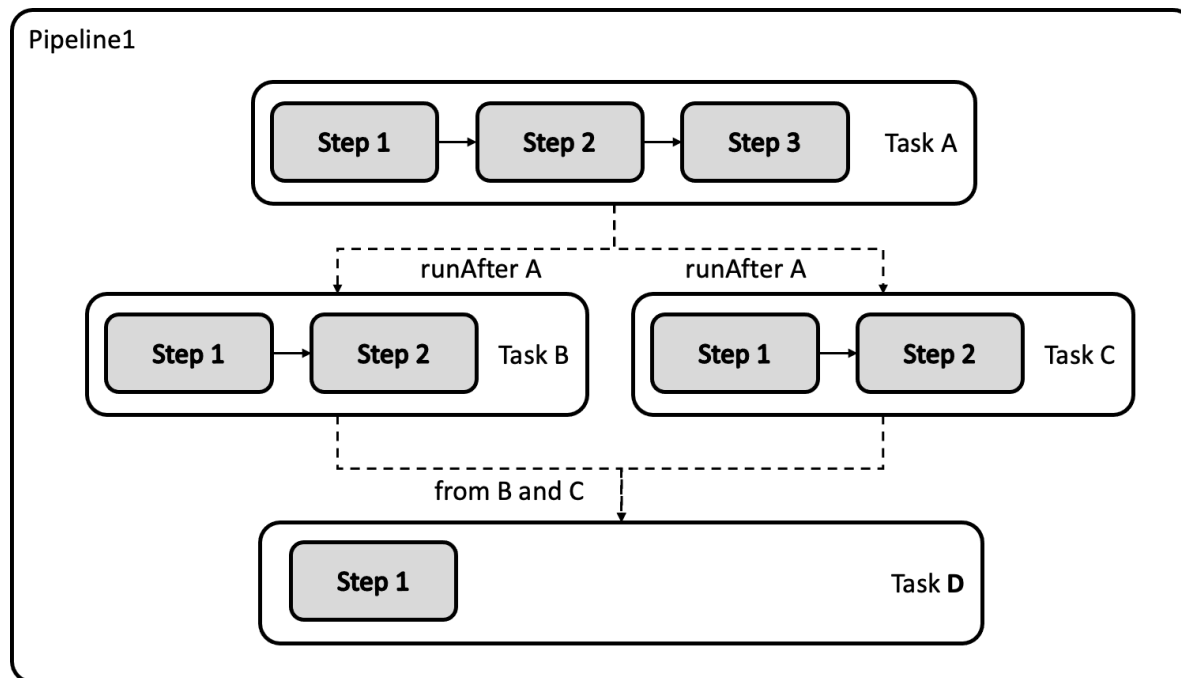
标准化：Tekton作为Kubernetes集群的扩展安装和运行，并使用业界公认的Kubernetes资源模型；Tekton作业以Kubernetes容器形态执行。

规模化支持：只需增加Kubernetes节点，即可增加作业处理能力。Tekton的能力可依照集群规模随意扩充，无需重新定义资源分配需求或者重新定义流水线。

Tekton核心组件

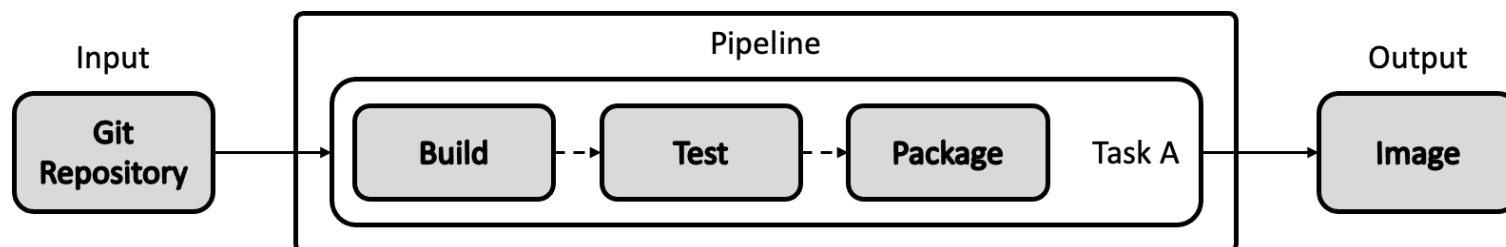
Pipeline: 对象定义了一个流水线作业，一个Pipeline对象由一个或数个Task对象组成。

Task: 一个可独立运行的任务，如获取代码，编译，或者推送镜像等等，当流水线被运行时，Kubernetes会为每个Task创建一个Pod。一个Task由多个Step组成，每个Step体现为这个Pod中的一个容器。

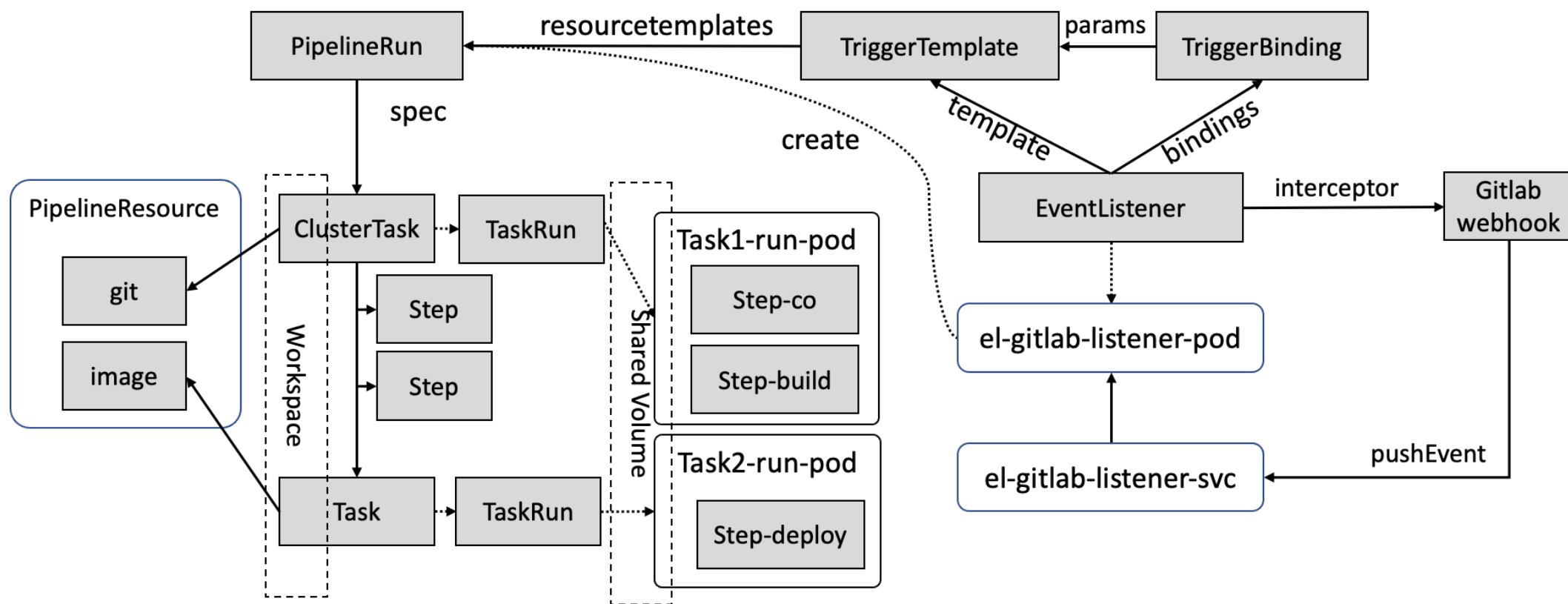


输入输出资源

Pipeline和Task对象可以接收git repository, pull request等资源作为输入，可以将Image, Kubernetes Cluster, Storage, CloudEvent等对象作为输出



事件触发的自动化流水线



EventListener

EventListener:

事件监听器，该对象核心属性是interceptors拦截器，该拦截器可监听多种类型的事件，比如监听来自GitLab的Push事件。

当该EventListener对象被创建以后，Tekton 控制器会为该EventListener创建Kubernetes Pod和Service，并启动一个HTTP服务以监听Push事件。

当用户在GitLab项目中设置webhook并填写该EventListener的服务地址以后，任何人针对被管理项目发起的Push操作，都会被EventListener捕获。

EventListener

```
apiVersion:
triggers.tekton.dev/v1alpha1
kind: EventListener
metadata:
  name: gitlab-listener
spec:
  serviceAccountName: cncamp-
gitlab-sa
  triggers:
    - name: gitlab-push-events-trigger
```

```
interceptors:
  - ref:
      name: gitlab
    params:
      - name: secretRef
        value:
          secretName: cncamp-gitlab-secret
          secretKey: secretToken
      - name: eventTypes
        value:
          - Push Hook
    bindings:
      - ref: gitlab-binding
  template:
    ref: gitlab-triggertemplate
```


TriggerTemplate

apiVersion: triggers.tekton.dev/v1alpha1

kind: TriggerTemplate

metadata:

name: gitlab-triggertemplate

spec:

params:

- name: gitrevision

- name: gitrepositoryurl

resourcetemplates:

- kind: PipelineRun

apiVersion: tekton.dev/v1beta1

metadata:

generateName: gitlab-pipeline-run-

spec:

serviceAccountName: cncamp-gitlab-sa

pipelineSpec:

tasks:

- name: checkout

taskRef:

name: gitlab-checkout

resources:

inputs:

- name: source

resource: source

resources:

- name: source

type: git

resources:

- name: source

resourceSpec:

type: git

params:

- name: revision

value: \$(tt.params.gitrevision)

- name: url

value: \$(tt.params.gitrepositoryurl)

GitLab Webhook

只需在GitLab中的被管理项目中设置一个Webhook，以确保该项目中的事件通知会发送至EventListener的服务地址即可。

Webhooks

[Webhooks](#) enable you to send notifications to web applications in response to events in a group or project. We recommend using an [integration](#) in preference to a webhook.

URL

URL must be percent-encoded if neccessary.

Secret token

Use this token to validate received payloads. It is sent with the request in the X-Gitlab-Token HTTP header.

Trigger

☒ **Push events**

URL is triggered by a push to the repository

7.Argocd

argocd

Argo CD是用于Kubernetes的声明性GitOps连续交付工具。

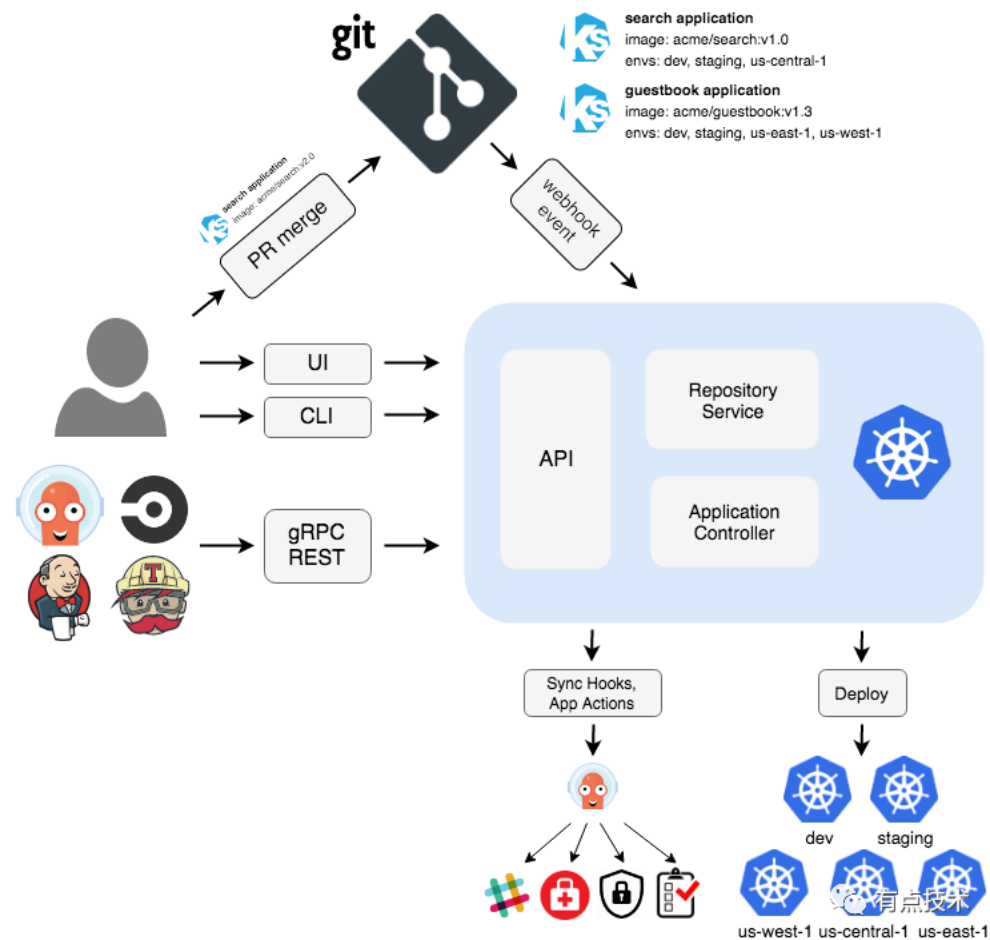
为什么选择Argo CD?

应用程序定义，配置和环境应为声明性的，并受版本控制。

应用程序部署和生命周期管理应该是自动化的，可审核的且易于理解的。

argo cd 架构

- Argo CD被实现为kubernetes控制器，该控制器连续监视正在运行的应用程序，并将当前的活动状态与所需的目标状态（在Git存储库中指定）进行比较。
- 其活动状态偏离目标状态的已部署应用程序被标记为OutOfSync。
- Argo CD报告并可视化差异，同时提供了自动或手动将实时状态同步回所需目标状态的功能。
- 在Git存储库中对所需目标状态所做的任何修改都可以自动应用并反映在指定的目标环境中。



安装argocd

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

确保 argocd-server service类型为NodePort

访问argocd 控制台

用户名: admin

密码: `k get secret -n argocd argocd-initial-admin-secret -oyaml`

argocd的适用场景

1. 低成本的Gitops利器
2. 多集群管理
 1. 不同目的集群：测试，集成，预生产，生产
 2. 多生产集群管理

8. 监控和日志

数据系统构建

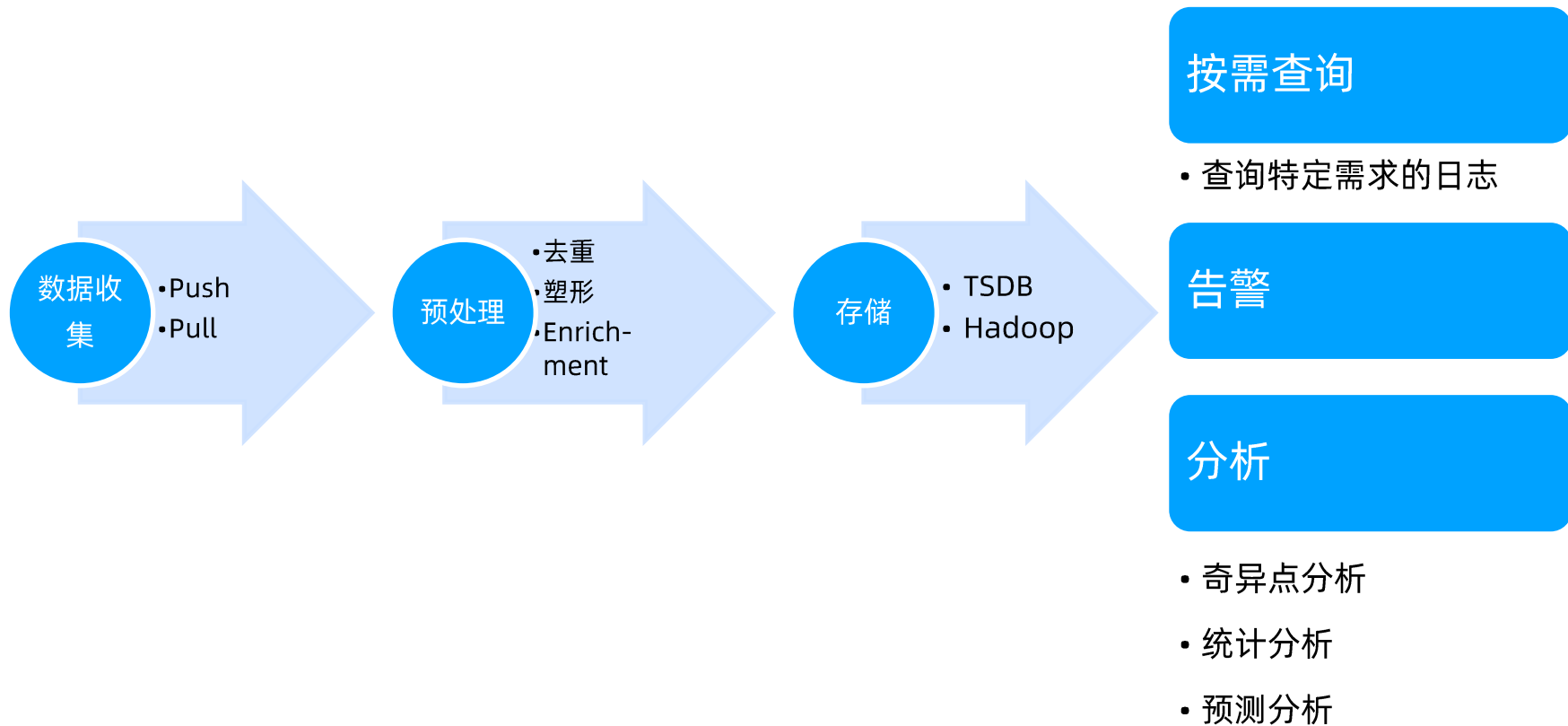
日志收集与分析

监控系统构建

日志系统的价值

- 分布式系统的日志查看比较复杂，因为多对节点的系统，要先找到正确的节点，才能看到想看的日志。日志系统把整个集群的日志汇总在一起，方便查看
- 因为节点上的日志滚动机制，如果有应用打印太多日志，如果没有日志系统，会导致关键日志丢失。
- 日志系统的重要意义在于解决，节点出错导致不可访问，进而丢失日志的情况。

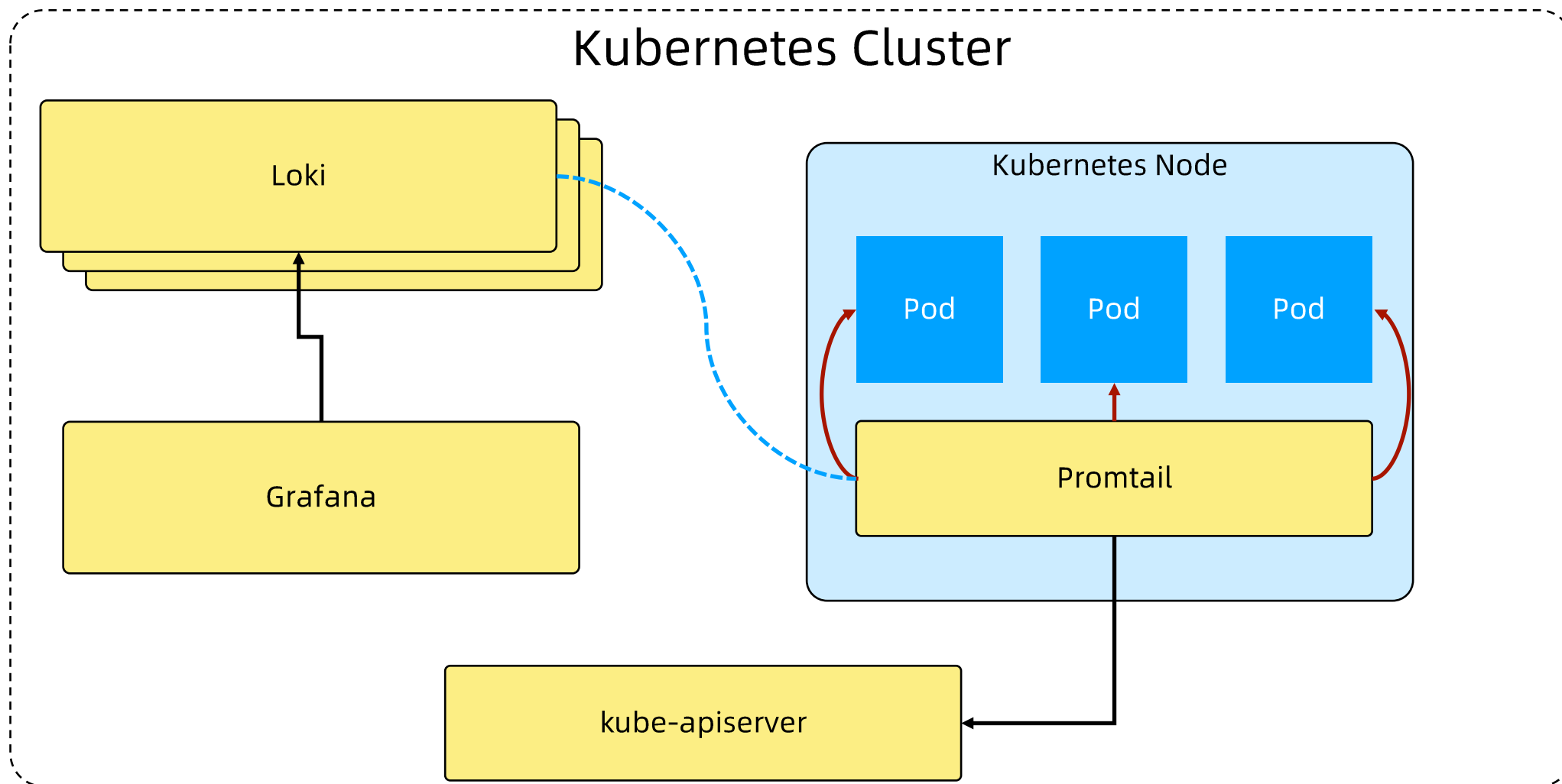
常用数据系统构建模式



日志收集系统Loki

- Grafana Loki是可以组成功能齐全的日志记录堆栈的一组组件。
 - 与其他日志记录系统不同，Loki是基于仅索引有关日志的元数据的想法而构建的：标签。
 - 日志数据本身被压缩并存储在对象存储（例如S3或GCS）中的块中，甚至存储在文件系统本地。
 - 小索引和高度压缩的块简化了操作，并大大降低了Loki的成本。

基于Loki 的日志收集系统



Loki-stack子系统

- Promtail

- 将容器日志发送到 Loki 或者 Grafana 服务上的日志收集工具
- 发现采集目标以及给日志流添加上 Label , 然后发送给 Loki
- Promtail 的服务发现是基于 Prometheus 的服务发现机制实现的, 可以查看configmap loki-promtail了解细节

- Loki

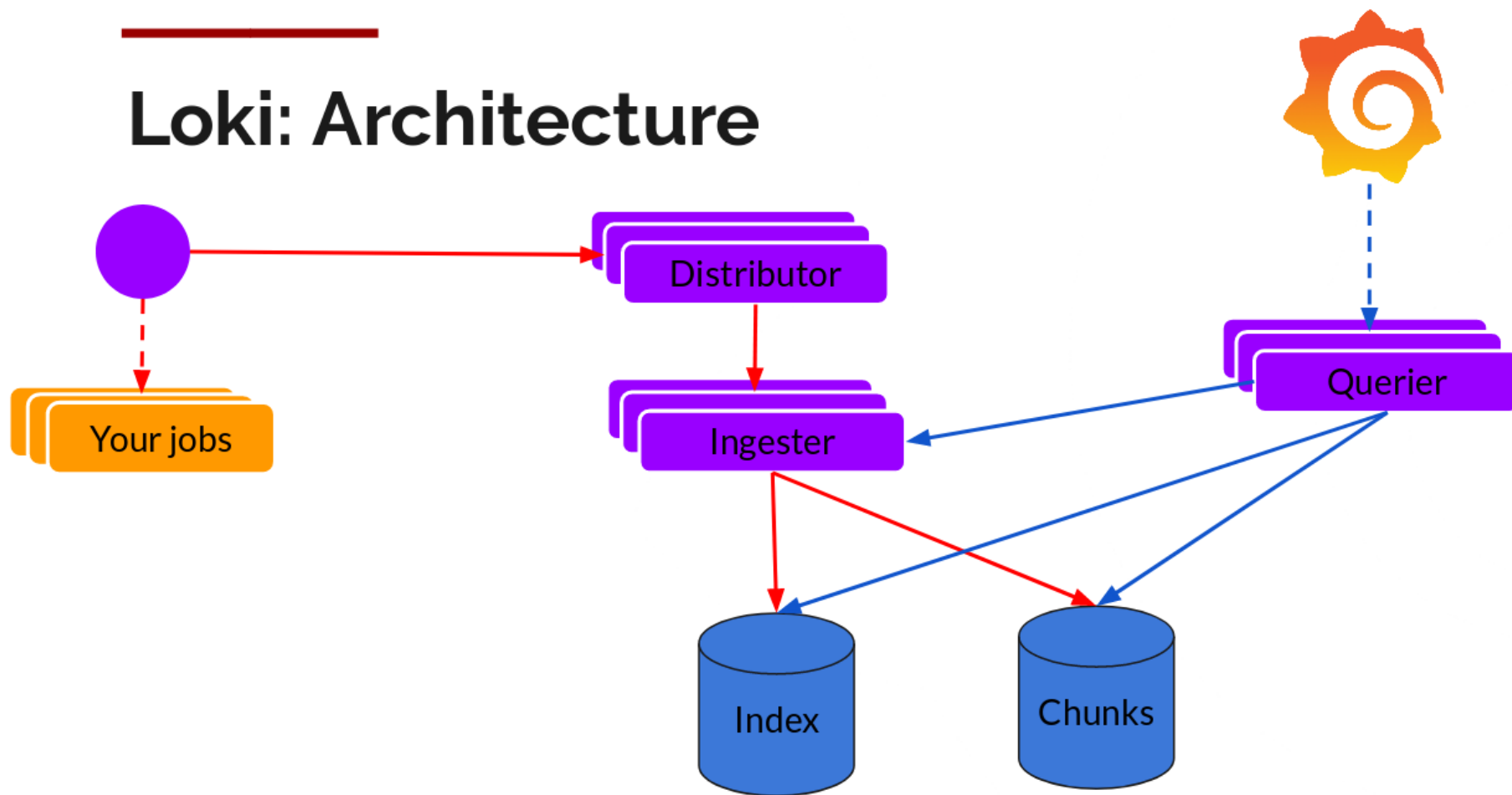
- Loki 是可以水平扩展、高可用以及支持多租户的日志聚合系统
- 使用和 Prometheus 相同的服务发现机制, 将标签添加到日志流中而不是构建全文索引
- Promtail 接收到的日志和应用的 metrics 指标就具有相同的标签集

- Grafana

- Grafana 是一个用于监控和可视化观测的开源平台, 支持非常丰富的数据源
- 在 Loki 技术栈中它专门用来展示来自 Prometheus 和 Loki 等数据源的时间序列数据
- 允许进行查询、可视化、报警等操作, 可以用于创建、探索和共享数据 Dashboard

Loki架构

Loki: Architecture



Loki组件

- **Distributor（分配器）**
 - 分配器服务负责处理客户端写入的日志。
 - 一旦分配器接收到日志数据，它就会把它们分成若干批次，并将它们并行地发送到多个采集器去。
 - 分配器通过 gRPC 和采集器进行通信。
 - 它们是无状态的， 基于一致性哈希， 我们可以根据实际需要对他们进行扩缩容。
- **Ingestor（采集器）**
 - 采集器服务负责将日志数据写入长期存储的后端（DynamoDB、S3、Cassandra 等等）。
 - 采集器会校验采集的日志是否乱序。
 - 采集器验证接收到的日志行是按照时间戳递增的顺序接收的， 否则日志行将被拒绝并返回错误。
- **Querier（查询器）**
 - 查询器服务负责处理 LogQL 查询语句来评估存储在长期存储中的日志数据。

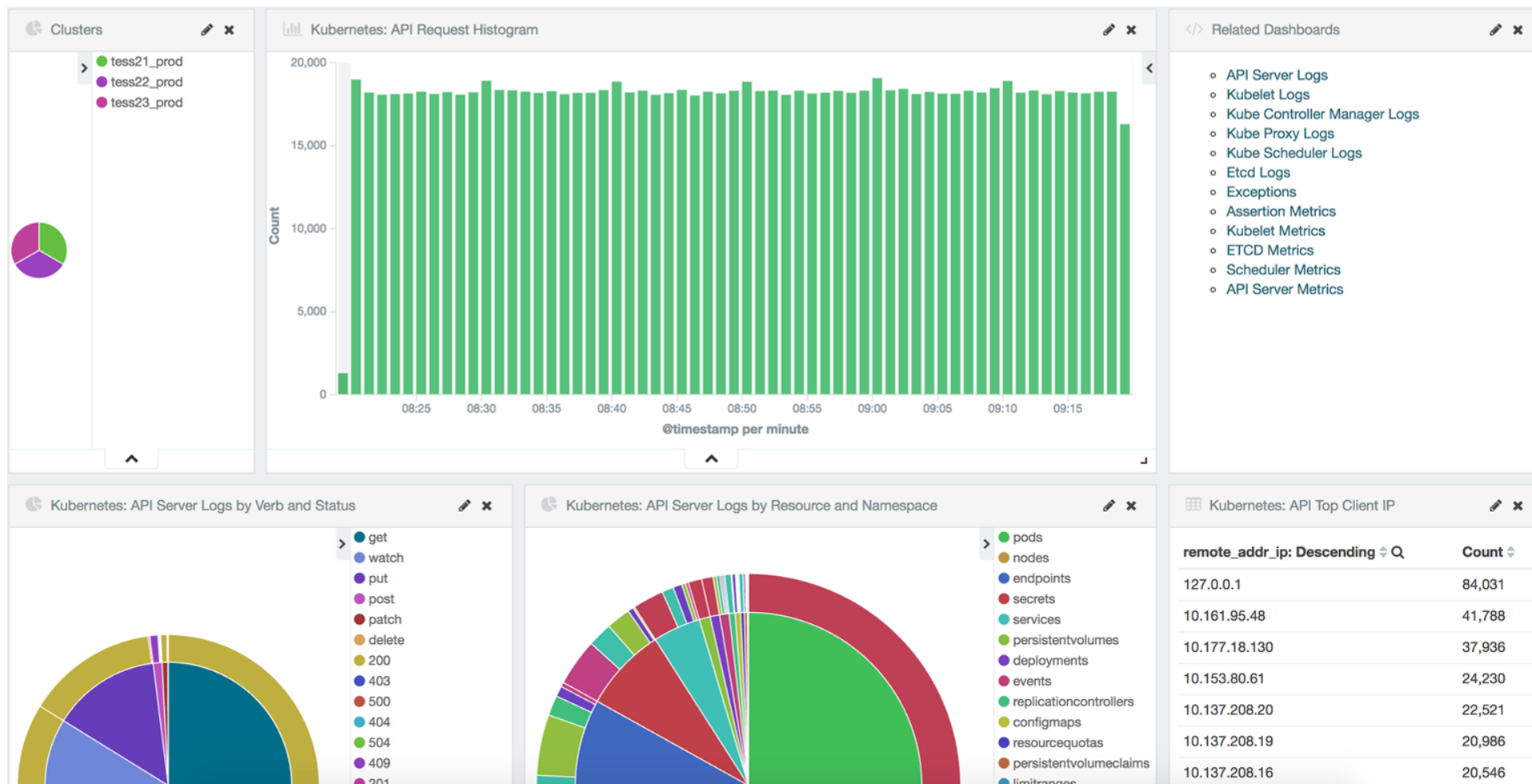
安装Loki stack

```
helm repo add grafana https://grafana.github.io/helm-charts
```

```
helm repo update
```

```
helm upgrade --install loki grafana/loki-stack --set  
grafana.enabled=true,prometheus.enabled=true,prometheus.alertmanager.persistentV  
olume.enabled=false,prometheus.server.persistentVolume.enabled=false
```

在Kubernetes集群中的日志系统



在生产中的问题

存在的问题

- 利用率低
 - 日志大多数目的是给管理员做问题分析用的，但管理员更多的是登陆到节点或者pod里做分析，因为日志分析只是整个分析过程中的一部分，所以很多时候顺手就把日志看了
- Beats出现过锁住文件系统，docker container无法删除的情况
- 与监控系统相比，日志系统的重要度稍低
- 出现过多次因为日志滚动太快而使得日志收集占用太大网络带宽的情况

监控系统

为什么监控，监控什么内容？

- 对自己系统的运行状态了如指掌，有问题及时发现，而不让用户先发现我们系统不能使用。
- 我们也需要知道我们的服务运行情况。例如，slowsql处于什么水平，平均响应时间超过200ms的占比有多少？

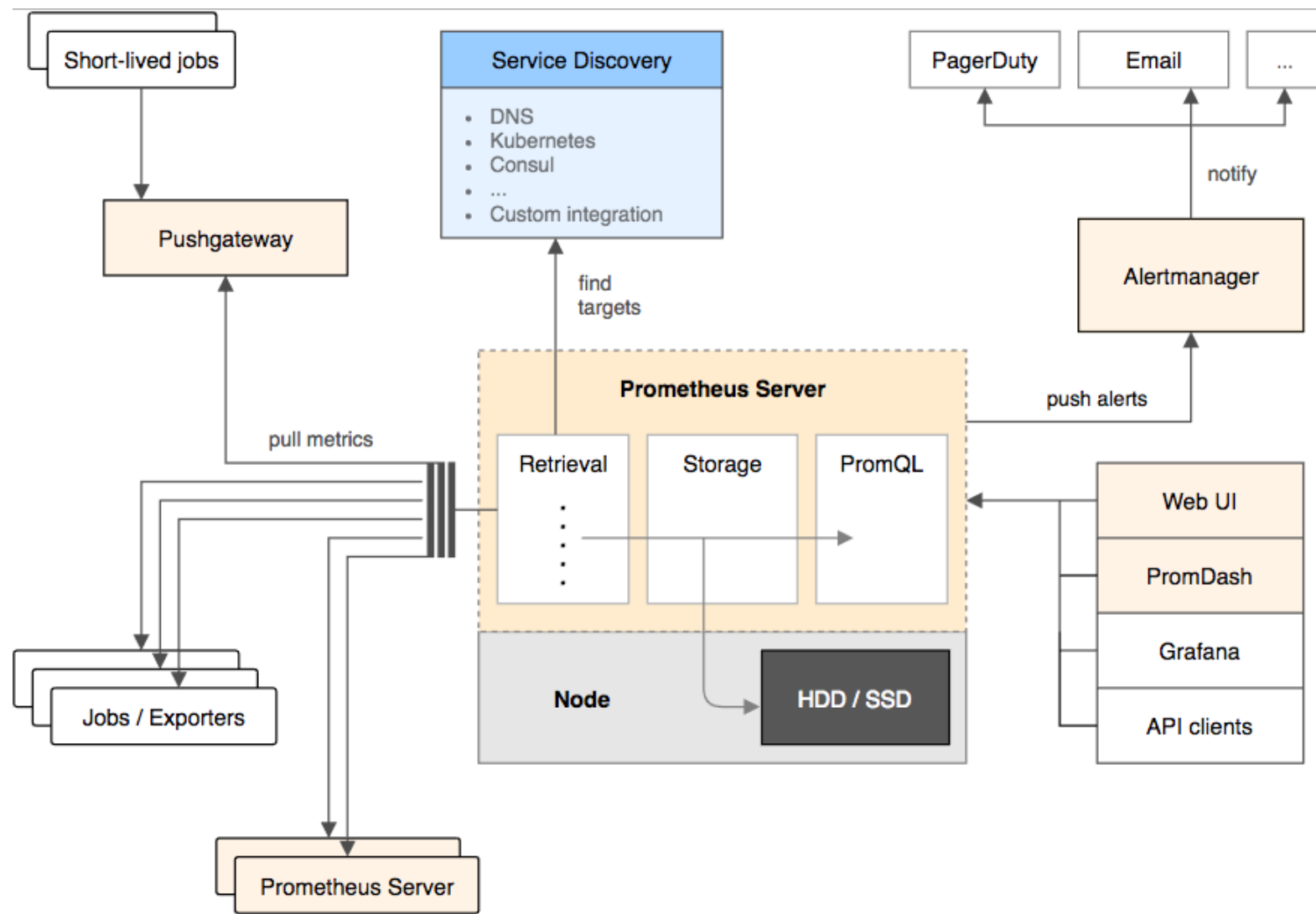
我们为什么需要监控我们的服务？

- 需要监控工具来提醒我服务出现了故障，比如通过监控服务的负载来决定扩容或缩容。如果机器普遍负载不高，则可以考虑是否缩减一下机器规模，如果数据库连接经常维持在一个高位水平，则可以考虑一下是否可以进行拆库处理，优化一下架构。
- 监控还可以帮助进行内部统制，尤其是对安全比较敏感的行业，比如证券银行等。比如服务器受到攻击时，我们需要分析事件，找到根本原因，识别类似攻击，发现没有发现的被攻击的系统，甚至完成取证等工作。

监控目的

- 减少宕机时间
- 扩展和性能管理
- 资源计划
- 识别异常事件
- 故障排除、分析

在Kubernetes集群中的监控系统



在Kubernetes集群中的监控系统

每个节点的kubelet（集成了cAdvisor）会收集当前节点host上所有信息，包括cpu、内存、磁盘等。Prometheus会pull这些信息，给每个节点打上标签来区分不同的节点。

```
# HELP container_cpu_system_seconds_total Cumulative system cpu time consumed in seconds.
# TYPE container_cpu_system_seconds_total counter
container_cpu_system_seconds_total{id="/" } 735292
container_cpu_system_seconds_total{id="/system.slice"} 710067.82
container_cpu_system_seconds_total{id="/system.slice/atd.service"} 0.04
container_cpu_system_seconds_total{id="/system.slice/auditd.service"} 652.29
container_cpu_system_seconds_total{id="/system.slice/cloud-config.service"} 0
container_cpu_system_seconds_total{id="/system.slice/cloud-final.service"} 0
container_cpu_system_seconds_total{id="/system.slice/cloud-init-local.service"} 0
container_cpu_system_seconds_total{id="/system.slice/cloud-init.service"} 0
container_cpu_system_seconds_total{id="/system.slice/crond.service"} 4267.6
container_cpu_system_seconds_total{id="/system.slice/dbus.service"} 3.44
container_cpu_system_seconds_total{id="/system.slice/dm-event.service"} 428.39
container_cpu_system_seconds_total{id="/system.slice/docker.service"} 55096.24
container_cpu_system_seconds_total{id="/system.slice/dracut-shutdown.service"} 0
container_cpu_system_seconds_total{id="/system.slice/fedora-autorelabel-mark.service"} 0
container_cpu_system_seconds_total{id="/system.slice/fedora-import-state.service"} 0
container_cpu_system_seconds_total{id="/system.slice/fedora-readonly.service"} 0
container_cpu_system_seconds_total{id="/system.slice/gssproxy.service"} 27.07
container_cpu_system_seconds_total{id="/system.slice/iscsi-shutdown.service"} 0
```

在Kubernetes中汇报指标

应用Pod需要声明上报指标端口和地址

apiVersion: v1

kind: Pod

metadata:

annotations:

prometheus.io/port: http-metrics

prometheus.io/scrape: "true"

name: loki-0

namespace: default

spec:

ports:

- containerPort: 3100

name: http-metrics

protocol: TCP

应用启动时，需要注册metrics

```
http.Handle("/metrics", promhttp.Handler())
```

```
http.ListenAndServe(sever.MetricsBindAddress, nil)
```

注册指标

```
func RegisterMetrics() {  
    registerMetricOnce.Do(func() {  
        prometheus.MustRegister(APIServerRequests)  
        prometheus.MustRegister(WorkQueueSize)  
    })  
}
```

代码中输出指标

```
metrics.AddAPIServerRequest(controllerName,  
    constants.CoreAPIGroup, constants.SecretResource,  
    constants.Get, cn.Namespace)
```

在Kubernetes集群中的监控系统

Kubernetes的control panel, 包括各种controller都原生的暴露Prometheus格式的metrics。

```
var (
    // TODO(a-robinson): Add unit tests for the handling of these metrics once
    // the upstream library supports it.
    requestCounter = prometheus.NewCounterVec(
        prometheus.CounterOpts{
            Name: "apiserver_request_count",
            Help: "Counter of apiserver requests broken out for each verb, API resource, client, and HTTP response contentType and code.",
        },
        []string{"verb", "resource", "client", "contentType", "code"},
    )
    requestLatencies = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name: "apiserver_request_latencies",
            Help: "Response latency distribution in microseconds for each verb, resource and client.",
            // Use buckets ranging from 125 ms to 8 seconds.
            Buckets: prometheus.ExponentialBuckets(125000, 2.0, 7),
        },
        []string{"verb", "resource"},
    )
    requestLatenciesSummary = prometheus.NewSummaryVec(
        prometheus.SummaryOpts{
            Name: "apiserver_request_latencies_summary",
            Help: "Response latency summary in microseconds for each verb and resource.",
            // Make the sliding window of 1h.
            MaxAge: time.Hour,
        },
        []string{"verb", "resource"},
    )
)
```


Prometheus中的指标类型

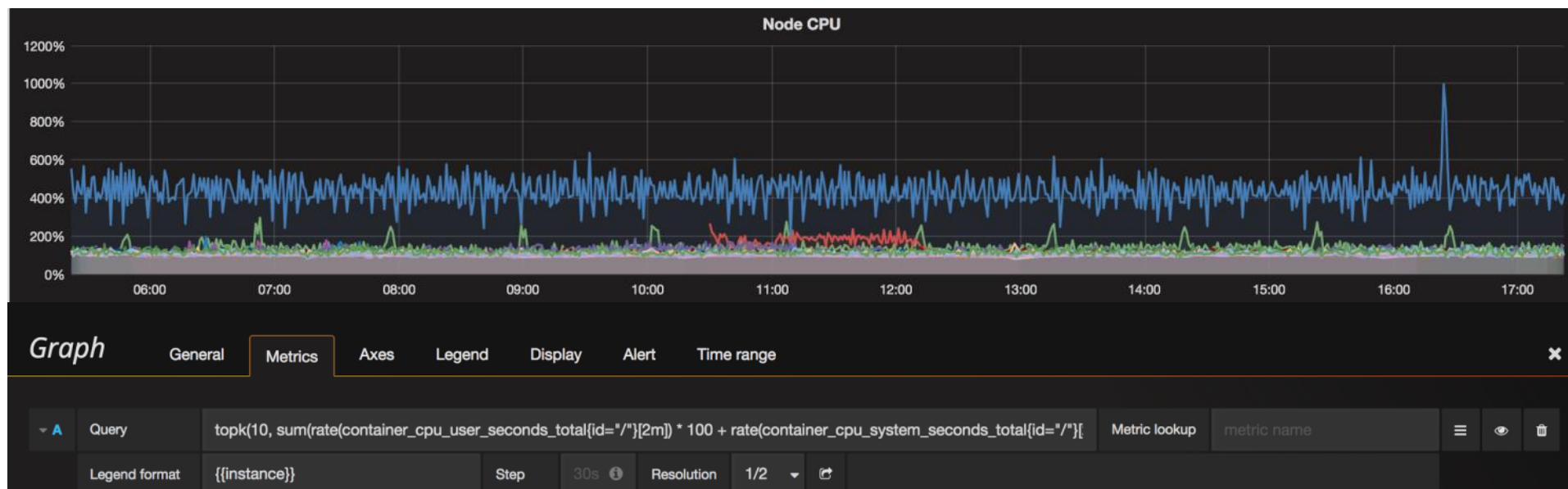
- Counter (计数器)
 - Counter 类型代表一种样本数据单调递增的指标，即只增不减，除非监控系统发生了重置。
- Gauge (仪表盘)
 - Gauge 类型代表一种样本数据可以任意变化的指标，即可增可减。
- Histogram (直方图)
 - Histogram 在一段时间范围内对数据进行采样（通常是请求持续时间或响应大小等），并将其计入可配置的存储桶（bucket）中，后续可通过指定区间筛选样本，也可以统计样本总数，最后一般将数据展示为直方图
 - 样本的值分布在 bucket 中的数量，命名为 `<basename>_bucket{le="<上边界>"}`
 - 所有样本值的大小总和，命名为 `<basename>_sum`
 - 样本总数，命名为 `<basename>_count`。值和 `<basename>_bucket{le="+Inf"}` 相同
- Summary (摘要)
 - 与 Histogram 类型类似，用于表示一段时间内的数据采样结果（通常是请求持续时间或响应大小等），但它直接存储了分位数（通过客户端计算，然后展示出来），而不是通过区间来计算
 - 它们都包含了 `<basename>_sum` 和 `<basename>_count` 指标
 - Histogram 需要通过 `<basename>_bucket` 来计算分位数，而 Summary 则直接存储了分位数的值。

Prometheus Query Language

- `histogram_quantile(0.95, sum(rate(httpserver_execution_latency_seconds_bucket[5m]))) by (le)`
- Histogram是直方图， `httpserver_execution_latency_seconds_bucket`是直方图指标，是将httpserver处理请求的时间放入不同的桶内，其表达的是落在不同时长区间的响应次数
- `by (le)`，是将采集的数据按桶的上边界分组
- `rate(httpserver_execution_latency_seconds_bucket[5m])`，计算的是五分钟内的变化率
- `Sum()`，是将所有指标的变化率总计
- 0.95，是取95分位
- 综上：
- 上述表达式计算的是httpserver处理请求时，95%的请求在五分钟内，在不同响应时间区间的处理的数量变化情况

在Kubernetes集群中的监控系统

Grafana Dashboard



开启告警

修改Prometheus配置文件prometheus.yml,添加以下配置:

rule_files:

- /etc/prometheus/rules/*.rules

在目录/etc/prometheus/rules/下创建告警文件hoststats-alert.rules内容如下:

groups:

- name: hostStatsAlert

rules:

- alert: hostCpuUsageAlert

expr: sum(avg without (cpu)(irate(node_cpu{mode!='idle'}[5m]))) by (instance) > 0.85

for: 1m

labels:

severity: High

annotations:

summary: "Instance {{ \$labels.instance }} CPU usage high"

description: "{{ \$labels.instance }} CPU usage above 85% (current value: {{ \$value }})"

构建支撑生产的监控系统

Metrics

- 收集数据

Alert

- 创建告警规则，如果告警规则被触发，则按不同眼中程度来告警

Assertion

- 以一定时间间隔，模拟客户行为，操作kubernetes对象，并断言成功，如果不成功则按眼中程度告警

来自生产系统的经验分享

Prometheus需要大内存和存储

- 最初prometheus经常发生OOM kill
- 在提高指定的资源以后，如果发生crash或者重启，prometheus需要30分钟以上的时间来读取数据进行初始化

Prometheus是运营生产系统过程中最重要的模块

- 如果prometheus down机，则没有任何数据和告警，管理员两眼一黑，什么都知道了

课后练习10.2

1. 为HTTPServer添加0-2秒的随机延时
2. 为HTTPServer项目添加延时Metric
3. 将HTTPServer部署至测试集群，并完成Prometheus配置
4. 从Prometheus界面中查询延时指标数据
5. （可选）创建一个Grafana Dashboard展现延时分配情况

THANKS

 极客时间 | 训练营